Cloud Security

SKY2100

Lecture script

A companion to the thirteen sessions

Timo Amling

Kristiania University of Applied Sciences

timo.amling@kristiania.no

Version 2.8 · August 2026

Preface

This script accompanies the course *SKY2100 Cloud Security*. Each chapter belongs to one session: Chapter 1 to Session 1, Chapter 2 to Session 2, and so on. The lecture slides follow the same structure.

The script is meant to be read before the session. It states in connected text what the slides sketch, works through the examples, and names at the end of every chapter what to read next in the course books. The books remain the curriculum: the CSA *Security Guidance v5* (CSA, 2024), Jøsang's *Cybersecurity: Technology and Governance* (Jøsang, 2024) and Comer's *The Cloud Computing Book* (Comer, 2021). Tanenbaum and Bos's *Modern Operating Systems* (Tanenbaum & Bos, 2015) supplies the operating-systems foundations of Sessions 2 and 7.

Three conventions apply. Every concept receives one definition, numbered and listed in the front matter, before it is used. Every session names a threat, judges its risk and derives a control in the vocabulary of Chapter 1. Citations give page numbers, so that every claim drawn from the course books can be checked at the source.

The SQ3R method fits the chapters: survey the headings and figures, formulate the questions the chapter should answer, read with those questions in mind, recite the answers in one's own words, and review after a few days. The self-check exercises support the last two steps.

Timo Amling, 2026

Contents

Preface **1**

List of abbreviations **11**

1 Cloud computing and information security **12**
1.1 Why cloud security . . . 12
1.2 Definition of cloud computing . . . 13
1.3 Security defined: the CIA triad . . . 13
1.4 Threat, risk and control . . . 14
1.5 Service models and shared responsibility . . . 15
1.6 Course design, literature and reading . . . 16
1.7 Summary . . . 16
Self-check . . . 17

2 Data centres, virtualisation and Linux **18**
2.1 The cloud as a distributed system . . . 18
2.2 Inside the data centre . . . 18
2.3 Traffic and geography . . . 18
2.4 Virtualisation and the hypervisor . . . 20
2.5 The Popek–Goldberg criterion . . . 20
2.6 Linux, the shell and the four pillars . . . 21
2.7 Laboratory: recording the baseline . . . 23
2.8 Summary . . . 23
Self-check . . . 23

3 Web services, load and denial of service **25**
3.1 Web services and capacity . . . 25
3.2 Measuring load . . . 25
3.3 Scaling out: load balancing and elasticity . . . 26
3.4 The minimum viable network . . . 27
3.5 Availability as a number: the SLA . . . 27
3.6 Denial of service . . . 28
3.7 Defending availability in layers . . . 28
3.8 Laboratory: Apache2 and Nginx under load . . . 29
3.9 Summary . . . 29
Self-check . . . 29

4 Networking and firewall controls **31**
4.1 Addresses, ports and the attack surface . . . 31
4.2 Networks in the cloud are software . . . 31
4.3 The firewall . . . 32
4.4 Firewall types, NGFW and WAF . . . 33

4.5 Defence in depth and Zero Trust . . . 34
4.6 Summary . . . 36
Self-check . . . 36

5 Cloud-native versus lift-and-shift **38**
5.1 Two ways into the cloud . . . 38
5.2 Containers . . . 38
5.3 The kernel . . . 39
5.4 Why containers, and orchestration . . . 40
5.5 Ephemeral versus immutable workloads . . . 40
5.6 Securing workloads in a moving environment . . . 41
5.7 The red thread: Nextcloud arrives . . . 42
5.8 Summary . . . 42
Self-check . . . 42

6 TLS and data in transit **44**
6.1 Three states of data . . . 44
6.2 Two families of cryptography . . . 44
6.3 TLS . . . 45
6.4 Beyond the browser: VPNs and data on the move . . . 46
6.5 Limitations of encryption . . . 46
6.6 Summary . . . 47
Self-check . . . 47

7 Hardening and access control **49**
7.1 Identity as the new perimeter . . . 49
7.2 Authentication and multi-factor authentication . . . 49
7.3 Authorisation: RBAC, PBAC and ABAC . . . 50
7.4 Protection domains and the access control matrix . . . 51
7.5 Single sign-on and privileged access . . . 51
7.6 Credentials and the management plane . . . 52
7.7 Summary . . . 53
Self-check . . . 53

8 DevSecOps, WAF and secure applications **54**
8.1 Application security in the cloud . . . 54
8.2 The secure development lifecycle . . . 55
8.3 DevSecOps . . . 55
8.4 The web application firewall . . . 56
8.5 Summary . . . 57
Self-check . . . 57

9 Security monitoring and IDS **59**
9.1 Monitoring in the cloud . . . 59
9.2 Events, alerts and telemetry . . . 59
9.3 Detection: from telemetry to alert . . . 60
9.4 Fast path and slow path . . . 61
9.5 Summary . . . 62
Self-check . . . 62

10 Logging and log integrity **63**
10.1 Log sources . . . 63
10.2 Logs, events and the content of a log line . . . 63
10.3 Central collection . . . 64
10.4 Log integrity . . . 65
10.5 Logs as evidence . . . 65
10.6 Summary . . . 66
Self-check . . . 66

11 Resilience, RAID and backup **68**
11.1 Availability revisited . . . 68
11.2 Redundancy and RAID . . . 68
11.3 Backup . . . 69
11.4 Redundancy the provider builds in . . . 70
11.5 Designing for failure . . . 70
11.6 Summary . . . 71
Self-check . . . 71

12 Incident response, BCM, risk and compliance **73**
12.1 Events, incidents and breaches . . . 73
12.2 The incident response lifecycle . . . 73
12.3 Resilience and business continuity . . . 75
12.4 Risk, audit and compliance . . . 75
12.5 Summary . . . 76
Self-check . . . 76

13 The integrated case **78**
13.1 The hardened stack . . . 78
13.2 Mapping to the NIST Cybersecurity Framework . . . 78
13.3 The argument . . . 79
13.4 Summary . . . 81
Closing exercise . . . 81

Reading list **82**

List of Figures

1.1 The CIA triad. Every control in this course protects at least one of the three properties; the recurring analytical question is which. 14

2.1 Two directions of traffic. North–south crosses the data-centre boundary; east–west stays inside it and is where lateral movement takes place (Session 4). 19

3.1 The wall. Response time holds flat while demand is below capacity and then bends sharply upward; the shape is the same whether the load is real customers or an attacker. 26
3.2 Scaling out. A load balancer spreads client requests across a cluster of identical servers, adding capacity, tolerating the failure of any one server, and letting the platform add or remove servers with demand. 26

4.1 A firewall under default-deny. Only traffic an explicit rule permits (here, the web ports) reaches the service; the final catch-all rule drops the rest, so anything unforeseen fails closed. 32
4.2 Which layers each tool operates at (filled = yes). Packet-filter and stateful firewalls act at Layers 3–4 (addresses, ports); an NGFW reaches up through Layer 7 with deep packet inspection; a WAF operates only at Layer 7, on the content of one web application. 34
4.3 Defence in depth. Independent layers around the asset, so that the failure of any one does not expose everything behind it. Each later session adds a ring. 35

5.1 A VM carries a whole guest operating system on virtualised hardware; a container is a packaged process sharing the host kernel: lighter, but with a thinner isolation wall. 39
5.2 Three kernels, one shape. All three enforce the same user/kernel split, reached only through system calls, but differ inside: Linux is *monolithic* (one privileged program), whereas Windows (NT) and macOS (XNU) are *hybrid*, layering an outer service layer over a small core. A Linux container's isolation is enforced by the Linux kernel, which is why it requires one. 40

6.1 The TLS handshake. The certificate authenticates the server and carries its public key; asymmetric cryptography then agrees a fresh symmetric session key, which encrypts the rest of the connection: slow handshake, fast conversation. 45

8.1 The DevSecOps loop. Development (plan, code, build, test) flows continuously into operations (release, deploy, operate, monitor) and back; DevSecOps embeds automated security at every stage rather than as a gate at the end, with SAST as the code is built and DAST against the deployed application. 56

9.1 IDS versus IPS. An IDS observes a copy of the traffic and alerts; an IPS sits in the path and can block. Detection is safe but passive; prevention acts but can break legitimate traffic. 60

10.1 Cascading log architecture. Each environment forwards to a central repository; the security-relevant subset is filtered into a retained audit store and fed to the SIEM, the same cascade-and-filter as in Session 9, now applied to storage and cost. 65

11.1 Three RAID levels. Striping (RAID 0) spreads blocks for speed with no redundancy; mirroring (RAID 1) keeps a full second copy; parity (RAID 5) distributes data and parity so that any one disk can be rebuilt. None of them is a backup. . . 69

12.1 The NIST incident response lifecycle. A cycle rather than a line: each incident's lessons feed back into preparation for the next. 74

12.2 The four phases with their cloud-specific work. Almost every action draws on a control built in an earlier session: preparation on Sessions 2, 9 and 11; the timeline on Session 10; containment on Sessions 5 and 7. 74

13.1 The hardened service. Each session added one independent layer to the same Nextcloud; together they form one defensible system. 79

List of definitions

1.1 Definition (Cloud computing, after NIST SP 800-145) . . . 13
1.2 Definition (The CIA triad) . . . 14
1.3 Definition (Threat, risk, control) . . . 14
1.4 Definition (Service models) . . . 15

2.1 Definition (Region, availability zone, region pair) . . . 19
2.2 Definition (Virtualisation, hypervisor) . . . 20

3.1 Definition (Web server) . . . 25
3.2 Definition (Capacity and availability) . . . 25
3.3 Definition (Service-level agreement) . . . 27
3.4 Definition (DoS and DDoS) . . . 28

4.1 Definition (Attack surface) . . . 31
4.2 Definition (Software-defined networking) . . . 32
4.3 Definition (Security group) . . . 32
4.4 Definition (Firewall, default-deny) . . . 32
4.5 Definition (Zero Trust) . . . 35

5.1 Definition (Lift-and-shift and cloud-native) . . . 38
5.2 Definition (Container) . . . 38
5.3 Definition (Ephemeral and immutable workloads) . . . 40

6.1 Definition (The three data states) . . . 44
6.2 Definition (Symmetric and asymmetric encryption) . . . 44
6.3 Definition (Certificate and PKI) . . . 45
6.4 Definition (TLS) . . . 45

7.1 Definition (Core IAM terms) . . . 49
7.2 Definition (Multi-factor authentication) . . . 50
7.3 Definition (Authorisation models) . . . 50
7.4 Definition (Least privilege) . . . 50

8.1 Definition (Secure development lifecycle) . . . 55
8.2 Definition (CI/CD and DevSecOps) . . . 55
8.3 Definition (Web application firewall) . . . 56

9.1 Definition (Event and alert) . . . 59
9.2 Definition (Intrusion detection and prevention) . . . 60

10.1 Definition (Log aggregation and SIEM) . . . 64
10.2 Definition (Log integrity) . . . 65

11.1 Definition (RAID) 68
11.2 Definition (Backup) 69

12.1 Definition (Event, incident, breach) 73
12.2 Definition (Incident response lifecycle) 73
12.3 Definition (Risk register) 75

List of exercises

1.1 Exercise (Five essential characteristics) . . . 17
1.2 Exercise (Strava case, in CIA terms) . . . 17
1.3 Exercise (Threat, risk, control for a vulnerability) . . . 17
1.4 Exercise (Open SSH port as threat–risk–control) . . . 17
1.5 Exercise (When SaaS reduces the risk) . . . 17

2.1 Exercise (North–south vs east–west traffic) . . . 23
2.2 Exercise (Type 1 vs Type 2 hypervisors) . . . 23
2.3 Exercise (Isolation as a security property) . . . 23
2.4 Exercise (Which command for which question) . . . 23
2.5 Exercise (Blast radius of one zone) . . . 23

3.1 Exercise (The load–security bridge) . . . 29
3.2 Exercise (Reading an ApacheBench command) . . . 29
3.3 Exercise (The three DDoS types) . . . 30
3.4 Exercise (Computing a composite SLA) . . . 30
3.5 Exercise (An application-layer flood) . . . 30

4.1 Exercise (Attack surface) . . . 36
4.2 Exercise (Default-deny reasoning) . . . 36
4.3 Exercise (A cloud firewall default) . . . 36
4.4 Exercise (NGFW vs WAF) . . . 36
4.5 Exercise (SSH brute force as threat–risk–control) . . . 36

5.1 Exercise (Lift-and-shift vs cloud-native) . . . 42
5.2 Exercise (Container vs VM isolation) . . . 42
5.3 Exercise (Why immutable workloads are safer) . . . 42
5.4 Exercise (A vulnerable dependency) . . . 42
5.5 Exercise (Why network scanning fails) . . . 42

6.1 Exercise (The three data states) . . . 47
6.2 Exercise (Why TLS uses both cipher types) . . . 47
6.3 Exercise (Trusting a server public key) . . . 47
6.4 Exercise (http to https as threat–risk–control) . . . 47
6.5 Exercise (Encrypted at rest, still stolen) . . . 47

7.1 Exercise (Authentication vs authorisation) . . . 53
7.2 Exercise (The three authentication factors) . . . 53
7.3 Exercise (Why the cloud needs MFA) . . . 53
7.4 Exercise (RBAC vs ABAC) . . . 53
7.5 Exercise (Shared admin account as threat–risk–control) . . . 53

8.1 Exercise (Shifting left) . . . 57

8.2 Exercise (SAST vs DAST) . . . 57
8.3 Exercise (What DevSecOps adds) . . . 57
8.4 Exercise (NGFW vs WAF, revisited) . . . 57
8.5 Exercise (A disclosed library flaw) . . . 57

9.1 Exercise (Event vs alert) . . . 62
9.2 Exercise (The two key telemetry sources) . . . 62
9.3 Exercise (Signature vs anomaly detection) . . . 62
9.4 Exercise (Fast path vs slow path) . . . 62
9.5 Exercise (The canary token) . . . 62

10.1 Exercise (The two roles of logs) . . . 66
10.2 Exercise (Central collection for ephemeral hosts) . . . 66
10.3 Exercise (Log integrity and the CIA triad) . . . 66
10.4 Exercise (Protecting log integrity) . . . 66
10.5 Exercise (Chain of custody) . . . 66

11.1 Exercise (Resilience vs prevention) . . . 71
11.2 Exercise (The RAID 0 trap) . . . 71
11.3 Exercise (Why RAID is not a backup) . . . 71
11.4 Exercise (The 3–2–1 rule) . . . 71
11.5 Exercise (Data loss as threat–risk–control) . . . 71

12.1 Exercise (Event, incident, breach) . . . 77
12.2 Exercise (The four incident-response phases) . . . 77
12.3 Exercise (Why rebuild rather than patch) . . . 77
12.4 Exercise (RTO and RPO) . . . 77
12.5 Exercise (Risk register and compliance inheritance) . . . 77

List of abbreviations

ABAC	Attribute-based access control
API	Application programming interface
BCM	Business continuity management
CIA	Confidentiality, integrity, availability
CSA	Cloud Security Alliance
CSF	(NIST) Cybersecurity Framework
CSP	Cloud service provider
CSPM	Cloud security posture management
DAST	Dynamic application security testing
DDoS	Distributed denial of service
DoS	Denial of service
DPI	Deep packet inspection
GRC	Governance, risk and compliance
HDFS	Hadoop Distributed File System
IaaS	Infrastructure as a service
IaC	Infrastructure as code
IAM	Identity and access management
IdM	Identity management
IDS	Intrusion detection system
SAST	Static application security testing
SBOM	Software bill of materials
SDN	Software-defined networking
SIEM	Security information & event mgmt
SOC	Security operations centre
SSDLC	Secure software development lifecycle
IPS	Intrusion prevention system
KMS	Key management service
MFA	Multi-factor authentication
NAS	Network-attached storage
NGFW	Next-generation firewall
NSG	Network security group (Azure)
OWASP	Open Web Application Security Project
PaaS	Platform as a service
PAM	Privileged access management
PBAC	Policy-based access control
PKI	Public key infrastructure
PoLP	Principle of least privilege
RAID	Redundant array of independent disks
RBAC	Role-based access control
RPO	Recovery point objective
RTO	Recovery time objective
SaaS	Software as a service
SAN	Storage area network
SASE	Secure access service edge
SSO	Single sign-on
TLS	Transport layer security
VM	Virtual machine
VNet	Virtual network (Azure)
VPN	Virtual private network
WAF	Web application firewall

Chapter 1

Cloud computing and information security

This chapter defines cloud computing, the three protection goals of information security, and the relation between threat, risk and control. Every later control (firewalls, TLS, access control, incident response) is described in this vocabulary.

1.1 Why cloud security

Phones, watches, cars and most business applications are online around the clock, and most of the services behind them run in the cloud. Industry forecasts put the data created worldwide in 2025 at roughly 180 zettabytes. More data and more services mean more exposure, and because most of these services run in somebody's cloud, most of that exposure sits there too.

Security failures are usually read as technical failures: a bug, a breach, a stolen password. The opening case has none of these and is still a security failure.

Example 1.1 (The Strava heat map)**.** In November 2017 the fitness platform Strava published an anonymised global heat map of its users' running and cycling routes: billions of aggregated activities without names. In most regions the map showed the expected pattern of parks and river banks. In January 2018 an analyst observed that in certain remote regions the only people wearing fitness trackers were soldiers, and their daily runs traced the perimeters and internal paths of military bases whose locations were classified.

No system was breached and no password was stolen. Every data point was anonymous, and the publication was a deliberate decision. Three lessons follow, and each returns throughout the course. Aggregated anonymous data can still reveal sensitive facts. The failure was a governance decision rather than a technical one, because somebody chose what to publish without reasoning about what could be inferred from it. And confidentiality is contextual, since the same route is harmless in a city park and sensitive at a military installation.

The case shows that cloud security is first a way of reasoning about who can see or change what, why, and what could go wrong. The reasoning connects a threat to a control that fits the risk and justifies the connection.

Comer (2021) names six factors that make security harder in the cloud than in a traditional infrastructure (Comer, 2021, pp. 233–234). The first is lack of control and visibility: the tenant cannot examine or configure the underlying systems and must trust the provider's configuration. The second is an infrastructure shared with outsiders. The third is the large number of services with dependencies among them, which enlarges the attack surface. The fourth is a constantly changing set of running instances, whose sudden growth is hard to tell apart from a denial-of-service attack. The fifth is remote access for all users, and the sixth is heavy use of software from

the provider and from third parties. Each factor returns in a later session: shared responsibility in this chapter, the dissolving perimeter in Session 4, the moving target in Session 5, remote identity in Session 7 and supply-chain risk in Session 8.

1.2 Definition of cloud computing

The cloud consists of physical machines in physical buildings, operated by an organisation that is usually not the user. What sets them apart from ordinary hosting is how their capacity is delivered: as a service, over a network, on demand. The definition below follows NIST Special Publication 800-145, which the CSA Security Guidance also adopts (CSA, 2024, pp. 12–14).

Definition 1.1 (Cloud computing, after NIST SP 800-145)**.** Cloud computing describes a model in which computing resources (networks, servers, storage, applications and services) are drawn from a shared pool, reached over a network, and set up and released on demand, quickly and with little effort on the part of the customer or the provider.

NIST names five essential characteristics, and a service that lacks one of them is hosting or outsourcing rather than cloud computing. The first is on-demand self-service: the customer provisions resources through a portal or an API, with no human on the provider's side. The second is broad network access: resources are reachable over standard networks from standard clients. The third is resource pooling: the provider's hardware serves many customers at once, so a virtual machine shares physical hardware with strangers. The fourth is rapid elasticity: capacity grows and shrinks quickly with demand. The fifth is measured service: usage is metered and paid for as consumed. Of the five, resource pooling produces most of the security questions in this course, because sharing hardware with strangers is exactly what a traditional data centre never did.

Example 1.2. Two ways of obtaining a server illustrate the five characteristics. In the classical way an organisation specifies hardware, obtains budget approval, orders, waits for delivery, racks, cables and installs; the process takes weeks to months. In the cloud way an administrator logs in to the provider's portal, chooses a size and a region and creates the machine. It answers to `ssh` within minutes and is metered by the hour. All five characteristics appear in the second scene: self-service (nobody approved anything), broad network access (the portal was reached from any location), resource pooling (the capacity was already standing by, shared), elasticity (one machine can become ten the same afternoon) and measured service (the bill covers exactly those hours). Session 2 repeats this scene in the laboratory. The security question of the whole course is contained in the same scene: the computer on which the server appeared belongs to somebody else.

The cloud is therefore somebody else's computer. This is not an argument against the cloud; it is the reason why cloud security is a shared problem. The provider owns the building, the power, the hardware and the virtualisation layer, and the customer owns what runs on top. Section 1.5 divides the layers between the two.

NIST also names four deployment models (CSA, 2024, pp. 15–16). A public cloud is offered to everyone, and this course uses one. A private cloud is operated for one organisation, a community cloud is shared by organisations with common requirements, and a hybrid cloud combines these. The security reasoning is the same in all four; only the allocation of responsibility changes with who operates the pool.

1.3 Security defined: the CIA triad

The cloud does not change what is protected; it changes who controls each layer on which protection is implemented. What is protected has three parts.

Definition 1.2 (The CIA triad)**.** Information security describes the protection of three properties of information and of the services that carry it. **Confidentiality** means that information is disclosed only to authorised parties, typically enforced by encryption and access control. **Integrity** means that information is not altered without authorisation, or at least not without detection, typically enforced by hashing, signing and logging. **Availability** means that information and services are reachable when needed, typically enforced by redundancy, resilience and protection against denial of service.

The triad structures the semester. Confidentiality is treated in Sessions 6 and 7 (encryption in transit, access control), integrity in Sessions 9 and 10 (monitoring, logging, the trustworthiness of the log record), and availability in Session 3 (load and denial of service) and Session 11 (resilience, RAID and backup). For this reason the first question in any scenario is which of the three properties is at stake.

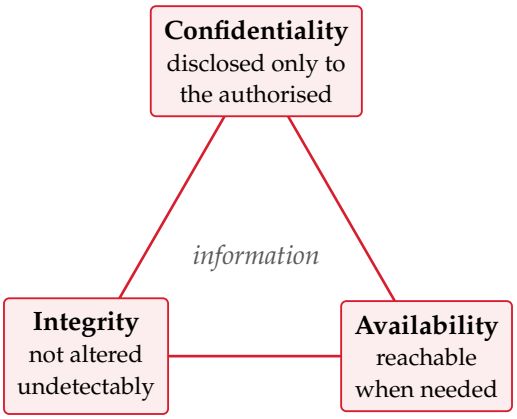

Figure 1.1. The CIA triad. Every control in this course protects at least one of the three properties; the recurring analytical question is which.

Example 1.3 (One file, three properties)**.** Throughout the course one concrete service is secured step by step: a *Nextcloud* file-sharing platform that the laboratory builds up over the semester, beginning with the Ubuntu machine beneath it, adding the service itself in Session 5 and defending it until Session 13. Consider a single file in it, `salary_list.pdf`. Confidentiality: only the finance staff may open it, and an employee who reads it without permission has violated confidentiality without touching anything else. Integrity: nobody may change a figure inside it unnoticed, and if a change occurs the log must show who made it and when. Availability: on the morning the payroll is run, the server must answer, so an attacker who merely makes Nextcloud unreachable has attacked the file without reading it. Three attacks, three defences, one file.

1.4 Threat, risk and control

In everyday language threat and risk are near-synonyms. In this course they are distinct terms, because the difference between them decides which control is appropriate.

Definition 1.3 (Threat, risk, control)**.** A **threat** describes a potential cause of an incident: something that could happen to the detriment of confidentiality, integrity or availability. **Risk** describes the weight of a threat, expressed as likelihood $\times$ impact. A **control** describes a measure that reduces a risk by making the threat less likely, its impact smaller, or both.

The three terms form a chain: name the threat, judge its likelihood and impact, and only then choose a control in proportion to that judgement. Many failures in practice break this order,

either because controls are bought before anyone has named the threat, or because threats are named without any judgement of likelihood.

Example 1.4 (Continuation of Example 1.2)**.** The virtual machine of Example 1.2 answers on its SSH port to the whole Internet. *Threat*: an attacker guesses or brute-forces the password of an account and logs in. *Risk*: the likelihood is high, because automated scanners find a fresh public SSH port within minutes, and the impact is high, because a login means full control of the machine. *Control*: key-based login only, and connections accepted only from the administrator's own address. Session 4 implements this control with firewall rules, and Session 7 generalises it into the principle of least privilege. The analysis of any incident follows the same structure: which CIA property was at stake, whose responsibility it was, and which single control would have prevented the incident.

1.5 Service models and shared responsibility

Every application sits on a stack: networking, storage, servers, virtualisation, operating system, middleware, runtime, and on top the customer's data and application. How much of the stack is rented decides which parts are the provider's problem and which remain the customer's. Three service models cover the common cases (CSA, 2024, pp. 16–20).

Definition 1.4 (Service models)**. On-premises** describes the arrangement in which everything runs on the organisation's own hardware, so that the organisation controls and secures the whole stack. **IaaS** (Infrastructure as a Service) describes the provision of fundamental compute, network and storage by the provider, with the customer managing everything from the operating system upward. **PaaS** (Platform as a Service) describes the additional provision of development and application platforms such as databases and runtimes, with the customer focusing on application and data. **SaaS** (Software as a Service) describes the provision of a complete hosted application, with the customer managing only configuration, identities and their own data.

Layer	On-prem	IaaS	PaaS	SaaS
Application	C	C	C	P
Data	C	C	C	C
Runtime & middleware	C	C	P	P
Operating system	C	C	P	P
Virtualisation	C	P	P	P
Servers & storage	C	P	P	P
Networking	C	P	P	P

Table 1.1. Responsibility by layer: C = customer, P = provider. From left to right the customer manages fewer layers; the data row never leaves the customer's column.

Table 1.1 is the *shared-responsibility model* (CSA, 2024, pp. 21–23), and two rules follow from it. First, responsibility follows control: whoever operates a layer secures it. In IaaS the provider therefore guards the building, the hardware and the hypervisor, whereas an unpatched operating system on the customer's virtual machine is the customer's incident. Secondly, the data row stays with the customer in every model. Even in SaaS, the customer decides who may access which data and what published data reveals, which is exactly the decision that failed in Example 1.1.

One further model appears in the learning outcomes. **SecaaS** (Security as a Service) describes security capabilities themselves, for example monitoring or identity services, consumed as a cloud service (CSA, 2024, p. 19). In the cloud, in other words, even the defences can be rented.

Example 1.5 (Continuation of Example 1.3)**.** The running example makes the table concrete. If Nextcloud runs on an IaaS virtual machine, as it does in the laboratory, the operating system, the web server, TLS, the firewall and every update are the customer's responsibility; the provider owes working hardware and isolation from the other tenants. If the organisation subscribes to a hosted SaaS file-sharing product instead, almost all of that moves to the provider. However, the decisions that would have prevented the Strava case, namely who may see what and what is shared with whom, remain with the customer. No service model therefore exists in which security is entirely somebody else's problem.

Resource pooling has one more consequence. Because a virtual machine shares physical hardware with strangers, the isolation between virtual machines is itself a security property: the provider delivers it and the customer must be able to trust it. Session 5 treats this isolation under workload security.

1.6 Course design, literature and reading

One service, the Nextcloud platform of Example 1.3, is the red thread of the semester. Sessions 1 and 2 prepare the machine beneath it, Session 3 installs Nginx, Session 5 installs the service with Docker, and each later week adds one layer: firewall, TLS, identity, WAF, monitoring, logging, backup. Session 13 then assembles the layers into one integrated case. Every session follows the same sequence of concept, threat, control, local laboratory, cloud transfer and reflection, and a typical week pairs a two-hour lecture with a two-hour laboratory in the students' own Ubuntu virtual machine. The laboratory of this first week is light: check that the computer supports hardware virtualisation, install VMware or VirtualBox, create an Ubuntu virtual machine, and complete the Microsoft Learn module *Describe cloud concepts*. Session 2 then covers Linux itself.

Nextcloud is a worked example, not a requirement: the method transfers to any self-hosted web service, not the product.

Three books carry the course, each with its own role. The *CSA Security Guidance v5* (CSA, 2024) provides the structure, since its twelve domains organise the sessions. Jøsang (2024) provides the security backbone: CIA, risk, cryptography, identity and access management, incident response and governance. Comer (2021) provides the cloud fundamentals: virtual machines, containers, Kubernetes and serverless computing. Tanenbaum and Bos (2015) supply the operating-systems foundations of Sessions 2 and 7. A short Microsoft Learn module accompanies each session, so Azure is the companion environment rather than a textbook.

Each session builds on terms the reading introduced, so the reading is best done before the session. The SQ3R method fits the chapters: survey the headings and figures, formulate the questions the chapter should answer, read with those questions in mind, recite the answers in one's own words, and review after a few days.

Reading for this chapter. CSA Security Guidance, Domain 1 (Cloud Computing Concepts & Architectures, pp. 10–27); Comer, Chapters 1–3 (pp. 5–32) and §17.2 (Cloud-Specific Security Problems, pp. 233–234); Jøsang, Chapter 1 (Basic Concepts of Cybersecurity, pp. 1–24). The Microsoft Learn module for Session 1 is linked on Canvas.

1.7 Summary

Cloud computing is a delivery model: on-demand network access to a pooled, elastic, metered set of computing resources (Definition 1.1), rented as IaaS, PaaS or SaaS (Definition 1.4). Security protects confidentiality, integrity and availability (Definition 1.2) by one chain of reasoning: name the threat, weigh the risk, then choose a control that fits the risk (Definition 1.3). What the cloud changes is the division of labour: responsibility follows control of each layer, no layer

is unowned, and the data, together with the judgement about what it reveals, stays with the customer (Example 1.1). Session 2 turns to the infrastructure beneath the model: the Linux server, its baseline and the virtualised data centre.

Self-check

Sketch answers follow the exercises.

Exercise 1.1 (Five essential characteristics)**.** Name the five essential characteristics of cloud computing and, for each, one sentence on where it appears in Example 1.2.

Exercise 1.2 (Strava case, in CIA terms)**.** In the Strava case (Example 1.1): which CIA property was harmed, and why is the claim that the data was anonymised not a sufficient defence?

Exercise 1.3 (Threat, risk, control for a vulnerability)**.** A team runs Nextcloud on an IaaS virtual machine. A critical vulnerability is published for the Linux distribution on that machine. Whose task is the patch, and which definition of this chapter answers the question?

Exercise 1.4 (Open SSH port as threat–risk–control)**.** Formulate the open SSH port of Example 1.4 strictly in the vocabulary of Definition 1.3: one sentence each for threat, risk and control.

Exercise 1.5 (When SaaS reduces the risk)**.** Give one realistic scenario in which moving from IaaS to SaaS reduces the security workload, and one responsibility that the move does not remove.

Sketch answers. (1) Self-service, broad network access, resource pooling, rapid elasticity, measured service: portal instead of ticket; created from anywhere; capacity was standing by, shared; one machine can become ten; billed by the hour. (2) Confidentiality of the bases, not of the individuals; aggregation re-identified what anonymisation hid, so the governance decision to publish was the failure. (3) The customer's: in IaaS the customer manages from the operating system upward (Definition 1.4, Table 1.1). (4) Threat: an attacker brute-forces SSH credentials. Risk: high likelihood (constant automated scanning) times high impact (full control). Control: key-only login restricted to the administrative address. (5) SaaS removes OS patching, TLS configuration and server hardening; it does not remove deciding who may access which data, because the data row stays with the customer.

Chapter 2

Data centres, virtualisation and Linux

This chapter covers the infrastructure beneath the cloud: data centres, virtualisation and the Linux server. Every later control attaches to one of these layers, so they are treated first. The laboratory records the baseline of the students' own virtual machine.

2.1 The cloud as a distributed system

The cloud is a *distributed system*: many computers in many buildings, cooperating over a network and presented to the customer as one service. Four properties follow from this. The first is scale: a large provider operates thousands to millions of servers that no human knows individually. The second is sharing: many customers, called *tenants*, use the same physical hardware, which is the resource pooling of Definition 1.1 under the name *multi-tenancy*. The third is failure as the normal case: at this scale some component is always broken, so the system is built to keep working regardless, and Session 11 applies the same attitude to the students' own designs. The fourth is abstraction: the customer rents capability without knowing which machine serves it.

Because a tenant shares hardware with strangers and cannot point at its own server, trust boundaries are logical rather than physical. They consist of layers, identities and policies instead of a locked door.

2.2 Inside the data centre

The data centre is a building engineered for density, power, cooling and resilience (Comer, 2021, pp. 37–42). Servers sit in *racks*, a top-of-rack switch connects the servers of each rack, and racks form *pods* for which power and cooling are engineered as a unit. Around the equipment sit redundant power with batteries and diesel generators, industrial cooling, physical access control and continuous monitoring.

Cooling consumes a large share of the energy, so the layout serves cooling: hot-aisle/cold-aisle containment places racks so that intake and exhaust air never mix. Because everything is managed remotely, modern data centres are lights-out buildings in which almost nobody works. Both facts bear on security, since physical access control and power-and-cooling redundancy are the foundation of availability (Definition 1.2). In Table 1.1 they are provider rows, and accepting those rows as P means trusting this building.

2.3 Traffic and geography

Traffic in a data centre has two directions (Comer, 2021, p. 44). *North–south* traffic runs between the data centre and the outside world, that is, clients calling services. *East–west* traffic runs between servers inside the data centre: services calling services, replication, remote storage

access. In the cloud, east–west traffic dominates, and this changes the network design. A tree-shaped network forces east–west traffic up the tree and back down, so the links near the root carry the combined load of everything beneath them, and because network hardware comes in fixed capacities, the root link cannot always be made fast enough (Comer, 2021, p. 45). The *leaf–spine* design removes the bottleneck (Comer, 2021, pp. 45–49): every leaf (top-of-rack) switch connects to every spine switch, so any server reaches any other over an equal-cost path. Where a single cable would not suffice, *link aggregation* bonds several physical links into one logical link, and a super spine extends the design further.

The same flat, any-to-any network that makes services fast also lets an intruder move sideways once inside, and because most cloud traffic is internal, that movement is invisible from outside. This *lateral movement*, an attacker moving from one internal system to the next, and the segmentation that limits it are the subject of Session 4.

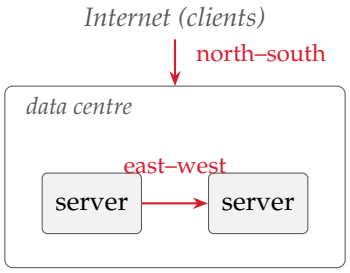

Figure 2.1. Two directions of traffic. North–south crosses the data-centre boundary; east–west stays inside it and is where lateral movement takes place (Session 4).

Providers also replicate capacity across the globe, so that a single failure or disaster does not take a customer down.

Definition 2.1 (Region, availability zone, region pair)**.** A **region** describes a geographic area containing one or more data centres (for example *Norway East*). An **availability zone** describes a physically separate data centre, or group of data centres, within a region, with independent power, cooling and network. A **region pair** describes two regions linked for replication and disaster recovery.

Zones bound the blast radius of a failure: a power problem in one zone leaves the others standing, so a deployment spread across zones survives it. Session 11 builds on this vocabulary.

Above the physical geography sits an administrative one. Azure organises resources in a hierarchy: management groups apply policy and access across many subscriptions; a subscription is a billing and isolation boundary; a resource group is a lifecycle container for related resources; and the resources themselves (a VM, a disk, a network) sit at the bottom. Governance, billing and access control all attach to this tree.

A small number of very large providers, which Comer (2021) calls *hyperscalers*, run data centres of this kind at global scale and rent their capacity to everyone else (Comer, 2021, pp. 31–32). The three largest are Amazon Web Services, Microsoft Azure and Google Cloud, and their dominance has two consequences. The first is *provider lock-in*: each provider's services and interfaces differ, so moving between them is costly. Large organisations therefore sometimes run *multi-cloud*, more than one provider on purpose, at the price of learning and securing two systems (Comer, 2021, pp. 28–31). The second is that the shared-responsibility model takes a different concrete form on each provider, so a control known on one platform, such as a security group's default, may behave differently on another. The concepts transfer; the buttons do not. Microsoft Azure is the worked environment of this course, and the reasoning built there carries to the others.

2.4 Virtualisation and the hypervisor

Virtualisation is what makes this infrastructure rentable.

Definition 2.2 (Virtualisation, hypervisor)**.** Virtualisation describes the technique by which one physical machine runs many isolated *virtual machines* (VMs), each behaving as if it owned the hardware. The **hypervisor** describes the software layer that mediates between them. A **Type 1** (bare-metal) hypervisor runs directly on the hardware (ESXi, Xen, Hyper-V) and is what data centres and clouds use. A **Type 2** (hosted) hypervisor runs as an application on a host operating system (VMware Workstation, VirtualBox) and is what the laboratory uses on the students' laptops.

Every rented cloud VM is a guest on a Type 1 hypervisor the customer never sees, and the Ubuntu machine in the laboratory is a guest on a Type 2 hypervisor the student installed. The mechanics are the same: the hypervisor gives each guest *virtual hardware*, namely a vCPU that is a share of real cores, a memory slice, a virtual disk that is a file on real storage, and a virtual network card attached to a virtual switch.

Isolation between guests rests on processor privilege levels (Comer, 2021, pp. 59–60). Normally the kernel runs privileged and applications run unprivileged. With virtualisation the hypervisor takes the most privileged level and each guest kernel moves one step down. The guest kernel behaves as if it controlled the hardware, but every privileged operation passes through the hypervisor, which also presents the emulated or para-virtualised devices. This layering is what keeps one VM from touching another.

Comer (2021) sorts the technologies into three approaches (Comer, 2021, pp. 55–57). Software emulation interprets every guest instruction in software; it is general but slow, and useful mainly when the guest was built for a different processor. Para-virtualisation modifies the guest operating system so that it cooperates with the hypervisor. Full virtualisation, the cloud default, runs the guest unmodified: almost all instructions execute directly on the real processor, and only privileged operations trap into the hypervisor. This is why the server of Example 1.2 appears in two minutes: creating a VM starts a program rather than assembling hardware.

2.5 The Popek–Goldberg criterion

A hypervisor must meet three requirements at once: *safety*, full control of the virtualised resources; *fidelity*, so that a program behaves on the virtual machine exactly as on bare hardware; and *efficiency*, so that most guest code runs without the hypervisor's intervention (Tanenbaum & Bos, 2015, p. 474). An interpreter that simulated every instruction would be safe and faithful but slow. Real hypervisors therefore let most instructions run directly on the processor, which raises the question of how the dangerous ones are caught.

Popek and Goldberg (1974) answered it (Tanenbaum & Bos, 2015, pp. 474–476). Every processor with a kernel mode and a user mode has *sensitive* instructions, which behave differently in the two modes (for example I/O or changes to the memory-management unit), and *privileged* instructions, which trap to the kernel when attempted in user mode. A machine is cleanly virtualisable only if the sensitive instructions are a subset of the privileged ones, because then every instruction that could reveal or disturb the real machine's state traps when a demoted guest attempts it, and the hypervisor can emulate the result. Trap-and-emulate, the mechanism behind full virtualisation, depends on this guarantee.

The x86 processor did not meet the criterion. Intel's 386 had sensitive instructions that were not privileged: `POPF` silently failed to change the interrupt-enable flag in user mode instead of trapping, and a program could read its code-segment selector to discover that it was in user mode. A guest kernel could therefore notice or misbehave without any trap, and for two decades the x86 could not host a hypervisor directly (Tanenbaum & Bos, 2015, p. 475). Two solutions

followed. VMware (from 1999) used *binary translation*: as the guest ran, its kernel code was rewritten on the fly, the unsafe instructions replaced by safe sequences that trapped into the hypervisor, while ordinary user code ran untouched. In 2005 Intel VT and AMD-V then added a processor mode in which trap-and-emulate works directly, controlled by a hypervisor-set bitmap of which operations trap. Modern cloud virtualisation runs on those extensions.

Two terms complete the picture (Tanenbaum & Bos, 2015, p. 477). A Type-1 hypervisor runs directly on the hardware with no host operating system beneath it, which is why cloud providers run this kind. A Type-2 hypervisor runs as an application inside a host operating system, which is what VMware Workstation and VirtualBox are on a laptop. Para-virtualisation avoids the trapping problem altogether by not pretending to be bare hardware: the guest is modified to call the hypervisor through *hypercalls*, explicit requests comparable to a program's system calls, and trades an unmodified guest for speed and simplicity.

Virtualisation thus works by forcing the guest's most dangerous actions to trap into a more privileged layer. A *VM escape* breaches that wall: code breaks out of its guest into the hypervisor, and every guest above it is exposed. Weak isolation shows first as the *noisy neighbour*, a tenant whose load degrades another's, and in the worst case as VM escape. The hypervisor is therefore the trust boundary between strangers sharing hardware, and Session 5 treats that boundary as something to be defended.

Example 2.1 (Continuation of Example 1.2)**.** Because a VM's hardware is virtual, its entire state (memory, disk, devices) is data; Comer (2021) calls the VM a *digital object* (Comer, 2021, pp. 63–64). The virtual machine of Example 1.2 can therefore be saved as a file, a *snapshot*, usable as backup, clone or roll-back point. It can also move while running: in *live migration* the hypervisor pre-copies the guest's memory to another physical host, pauses the guest briefly, finalises the transfer, and the machine continues elsewhere with almost no downtime (Comer, 2021, pp. 64–65). Providers use this constantly to balance load and to service hardware without taking customers offline.

The security implication follows from the VM being a computer reduced to data: a VM-as-file can be copied or stolen as a whole, and a snapshot may capture secrets that were only ever meant to exist in memory. Data security (Sessions 6 and 10) inherits this problem.

VMs are not the only packaging. A *container* virtualises the operating system rather than the hardware: it shares the host's kernel, weighs megabytes instead of gigabytes and starts in seconds, at the price of weaker isolation (Comer, 2021, pp. 71–72). A VM is a whole computer in software, whereas a container is a packaged process, and the two therefore differ in threat model. Session 5 covers containers.

2.6 Linux, the shell and the four pillars

Almost all cloud infrastructure runs on Linux, a UNIX-like kernel first released by Linus Torvalds in 1991, for four practical reasons. It is open source, so there is no per-server licence at scale and the code can be inspected and modified. It is stable and modular, running for years with only the components installed that are needed. It is scriptable, since everything is reachable from the command line, which is what makes automation possible. And it is everywhere, from supercomputers to phones and routers. Every laboratory from here on therefore runs in the students' Ubuntu VM; Ubuntu is the *distribution* used here, the kernel plus a chosen set of tools.

Linux, macOS and Windows share the parts that security reasoning rests on (Tanenbaum & Bos, 2015, Chapters 10 and 11). All three enforce the kernel/user split of Section 2.4, give every process its own virtual address space, schedule processes pre-emptively and route all hardware access through system calls. Their kernels differ in structure: Linux has a *monolithic* kernel, one large privileged program that loads modules; Windows uses a layered *hybrid* NT kernel; macOS runs XNU, a hybrid of the Mach microkernel and a BSD UNIX layer. Because

the shared part dominates, the protection-domain model of Session 7 applies to all three. Two differences matter in practice. Linux and macOS are UNIX-like and POSIX-compatible, so their shells, permissions and tools are close, whereas Windows has its own APIs. And the public cloud runs mainly on Linux servers, which are free to run at scale and are the platform almost all cloud software targets, whereas Windows Server holds an enterprise niche and macOS is a desktop system. This course works in Linux because the servers to be defended run it.

A server has no graphical desktop, so work runs through a *shell*, usually `bash`. Four commands suffice for orientation: `pwd` (current directory), `ls -l` (list files with permissions), `cd` (change directory) and `cat` or `less` (read a file). The filesystem is one tree, and the Filesystem Hierarchy Standard fixes where things live: configuration under `/etc`, log files under `/var/log`, user files under `/home`, programs under `/bin` and `/usr`. Session 7 covers users and permissions.

Linux is multi-user, and one account is special: `root`, which the kernel does not refuse. Working permanently as root means that any slip, or any attacker who hijacks the session, acts with unlimited impact. The convention is therefore to work as an ordinary user and to raise single commands with `sudo`, which asks for a password and leaves a trace; this is least privilege (Session 7) in tool form. Software is installed and updated through the package manager: `sudo apt update` refreshes the catalogue and `sudo apt upgrade` applies pending updates. On an IaaS machine this patching is the customer's job, and an unpatched service is among the most common ways real systems are compromised.

The chapter ends on the command line because of a principle that carries the second half of the course: before something abnormal can be recognised, normal must be known. A baseline of CPU, memory, disk and network is therefore the starting point of all detection.

Resource	**Command**	**Output**
CPU	`lscpu`	model, cores, virtualisation flags
CPU / load	`top` / `htop` / `btop`	live processes, CPU %, load average
Memory	`free -h`	used and free RAM and swap
Disk	`df -h`	free space per filesystem
Disk	`du -sh` *dir*	size of a directory
Network	`ss -tulpn`	listening sockets and open ports
Logs	`journalctl`	system and service log entries

Table 2.1. The observation commands used in every laboratory of the semester.

The table has four pillars (CPU, memory, disk, network) plus logs. `top` and its successors show live load, and the *load average* compares demand to the number of cores. `free -h` shows whether the system is swapping, a sign of memory pressure. A full disk takes services down, so `df` and `du` matter more than they appear to. `ss -tulpn` lists every port the machine listens on, and each of them is a door an attacker may try, which Session 4 turns into policy. `journalctl`, lastly, is the primary log source for Session 10.

Example 2.2. Three observations show why a baseline is a security tool and not only an administrator's habit. A sudden CPU spike at 03:00 may be crypto-mining, a runaway process or an attack. A disk filling unusually fast may be logs flooding, or data being staged for exfiltration. An unexpected listening port is a service nobody started. None of these observations answers anything by itself, but each of them, compared against a known baseline, shows where the threat–risk–control questions of Definition 1.3 should be asked. Without the baseline the deviation would not be noticed. In Sessions 9 and 10 exactly these signals feed an intrusion detection system and the incident timeline; monitoring is the same set of metrics observed continuously.

2.7 Laboratory: recording the baseline

Exercise 2 has three steps. First, boot the Ubuntu VM prepared after Session 1, open a terminal and inspect the system with `lscpu`, `free -h`, `df -h`, `ss -tulpn` and `journalctl -xe`. Secondly, install `htop` and `btop` with `sudo apt install` and compare them with plain `top`. Thirdly, record the output, label it and keep it, because this recording is the reference against which every later week measures change.

Reading for this chapter. Comer, Chapter 4 (Data Center Infrastructure and Equipment, pp. 37–51) and Chapter 5 (Virtual Machines, pp. 55–68), the primary reading this week; CSA Security Guidance, Domain 7, Section 7.1 (Cloud Infrastructure Security, pp. 147–153). For the theory of virtualisation in depth: Tanenbaum and Bos, *Modern Operating Systems*, §7.2 (Requirements for Virtualization, the Popek–Goldberg criterion, pp. 474–476) and §7.3 (Type 1 and Type 2 Hypervisors, p. 477); for the operating-system families, Chapter 10 (UNIX, Linux and Android) and Chapter 11 (Windows). On providers and lock-in: Comer, Chapter 3 (Types of Clouds, including hyperscalers and multi-cloud, pp. 25–32). Optional deepening: Jøsang, Chapter 3 (System Security, incl. virtualisation in §3.6, pp. 43–62). The Microsoft Learn module is *Describe Azure architecture and services*; students map regions, zones, subscriptions and resource groups onto the portal.

2.8 Summary

The cloud is a distributed system: many shared machines in engineered buildings, where failure is normal and trust boundaries are logical. Scale is organised in racks and pods, in regions, availability zones and region pairs (Definition 2.1), and in a management hierarchy. A hypervisor (Definition 2.2) gives each guest virtual hardware and passes every privileged operation through itself; the Popek–Goldberg criterion states when that works, and Intel VT and AMD-V made it work on x86. Isolation between strangers' workloads is therefore a security property. Because a machine is data, snapshots and live migration are possible, and so are their risks (Example 2.1). Linux runs the cloud, the shell is the interface, and the four pillars of Table 2.1, measured while the system is healthy, are the baseline for every later anomaly. Session 3 places a real service on the machine and pushes it until it strains.

Self-check

Sketch answers follow the exercises.

Exercise 2.1 (North–south vs east–west traffic)**.** Explain north–south versus east–west traffic, state which dominates in the cloud, and name the network design that answers it.

Exercise 2.2 (Type 1 vs Type 2 hypervisors)**.** Define both hypervisor types, give one example of each, and state where each appears in the course setup.

Exercise 2.3 (Isolation as a security property)**.** Why is isolation between virtual machines a security property and not merely a performance concern? Name the mild and the worst-case failure of it.

Exercise 2.4 (Which command for which question)**.** Which command answers each question: Which ports is this machine listening on? How much memory is free? What has the `ssh` service logged?

Exercise 2.5 (Blast radius of one zone)**.** A deployment runs in one availability zone of one region. Which failures in Definition 2.1 does it survive, and what should be changed first?

Sketch answers. (1) North–south: data centre ↔ outside world; east–west: server ↔ server inside. East–west dominates (services call services, replication, remote storage); leaf–spine fabrics give any-to-any capacity. (2) Type 1 runs on bare metal (ESXi, Xen, Hyper-V), beneath every rented cloud VM; Type 2 runs on a host OS (VMware Workstation, VirtualBox), beneath the laboratory's Ubuntu VM. (3) Because tenants are strangers sharing hardware and only hypervisor mediation separates them. Mild failure: noisy neighbour; worst case: VM escape, after which every guest is exposed. (4) `ss -tulpn`; `free -h`; `journalctl` (for example `journalctl -u ssh`). (5) It survives failures inside other zones only; a zone-level failure (power, cooling, network of its own zone) takes it down. First change: spread across availability zones; regional disasters then motivate the region pair.

Chapter 3

Web services, load and denial of service

This chapter places a web server on the measured machine of Chapter 2, loads it until it strains, and reads the strain with the baseline tools. The subject is availability. Because a denial-of-service attack is demand engineered to exceed capacity, understanding load is the start of the defence.

3.1 Web services and capacity

Definition 3.1 (Web server)**.** A **web server** describes a program that listens on a network port, accepts HTTP *requests* and returns *responses* (pages, data or files). A request carries a method (`GET`, `POST`, ...), a path and headers. A response carries a status code (`200` OK, `404` not found, `503` service unavailable, ...), headers and a body.

The web server of this course is *Nginx*: fast, widely deployed, usable as a reverse proxy and, from Session 8, as the host of the web application firewall. The laboratory installs it this week. The distance between a `200` and a `503` response is what load measurement makes visible.

A single machine answers many users through *concurrency*, that is, by handling many connections at the same time, and three facts set the limit. Each client occupies a connection while it is served, and the number of connections is finite. The server runs several *workers*, each handling many connections. Every request costs CPU, memory and I/O. When requests therefore arrive faster than the server completes them, the queue grows, latency rises and requests are dropped. That ceiling is *capacity*.

Definition 3.2 (Capacity and availability)**. Capacity** describes the amount of work a system can do per unit time (for a web server: requests per second), bounded by CPU, memory, connections and bandwidth, and established by measurement rather than estimation. **Availability**, one of the three CIA properties, describes the service being reachable and responsive when needed. Availability is lost when demand exceeds capacity, whether by accident or by attack.

Because a denial-of-service attack is demand engineered to exceed capacity, the next section covers measurement before it covers attackers.

3.2 Measuring load

Capacity is established by pushing the service until it strains, on one's own machine. The tool is `ab` (ApacheBench), which sends many requests at a chosen concurrency:

```
ab -n 1000 -c 50 http://localhost/
```

This sends one thousand requests, fifty at a time, and reports requests per second, time per request and the share of requests that failed. While `ab` runs, `htop` or `btop` in a second terminal shows CPU and memory climbing, and Nginx writes every request to the access log at `/var/log/nginx/access.log`. These are the tools of Table 2.1, now pointed at a service under pressure.

Example 3.1 (Continuation of Example 2.2)**.** At low concurrency, latency barely moves: the service degrades *gracefully*. As concurrency rises, queues build and time per request climbs. Beyond a certain point the curve bends sharply upward, timeouts appear, `503` responses replace `200` responses, and the service is effectively down. Plotting response time against concurrent requests yields a flat region followed by a steep rise, which this script calls the wall. Two observations follow. First, the wall is a property of a specific service on a specific machine. It can be measured in an afternoon, and it is preferable to know it before someone else finds it. Secondly, the shape is the same whether the load comes from real customers or from an attacker. The metrics cannot show why demand exceeded capacity; they show only that it did. Distinguishing stress from attack requires the logs and the methods of later sessions.

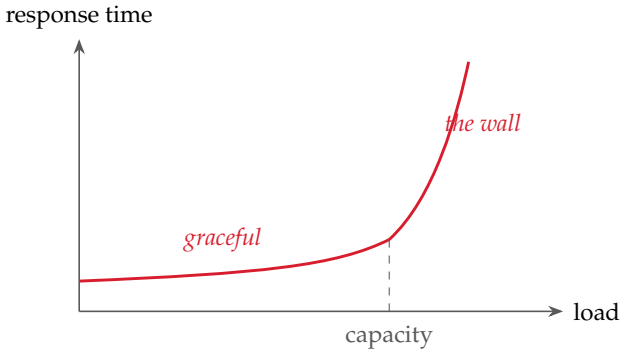

Figure 3.1. The wall. Response time holds flat while demand is below capacity and then bends sharply upward; the shape is the same whether the load is real customers or an attacker.

3.3 Scaling out: load balancing and elasticity

A single server has a hard ceiling, and the cloud grew out of one early answer to it. Popular web sites of the early Internet exceeded the capacity of one machine, so a *cluster* of identical servers was placed behind a *load balancer* that spreads requests across them (Comer, 2021, pp. 8–9). Scaling *out* with many commodity machines proved cheaper and more resilient than scaling *up* to one ever-larger machine, and shared buildings made the model affordable (Comer, 2021, pp. 10–11). Load balancing is therefore one of the ideas the cloud is built on.

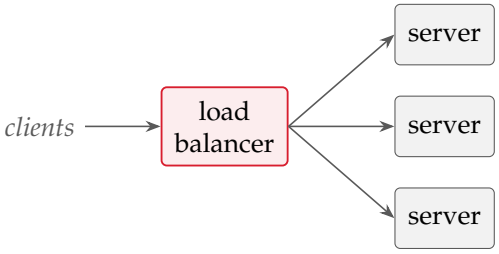

Figure 3.2. Scaling out. A load balancer spreads client requests across a cluster of identical servers, adding capacity, tolerating the failure of any one server, and letting the platform add or remove servers with demand.

A load balancer provides three things at once. It provides capacity, since more servers share the work. It provides resilience, since one server can fail and the balancer stops sending it traffic. And it enables *autoscaling*, since interchangeable instances let the platform add servers when load rises and remove them when it falls. This third property is the rapid elasticity of Definition 1.1: the customer pays for the peak only while the peak lasts, and the provider re-sells the idle capacity.

Comer (2021) calls this *elastic computing* and traces it to virtualisation (Comer, 2021, pp. 16–18). Because a virtual server is created and destroyed by software in moments, the provider can follow each tenant's demand and pack many tenants' fluctuating loads onto the same physical machines, since their peaks rarely coincide. The customer changes scale without owning hardware for the maximum, and the provider keeps its machines busy. The same rapid appearance and disappearance of instances, however, turns the attack surface into a moving target, which Session 5 takes up.

Azure offers the corresponding services, and this week's Microsoft Learn module covers them. A *Virtual Network* (VNet) is the customer's private network in the cloud, and its segmentation limits the blast radius of an incident. The *Azure Load Balancer* distributes traffic within a region at Layer 4 (IP address and port), whereas the *Application Gateway* balances at Layer 7 and hosts the WAF of Session 8. *Traffic Manager* and *Front Door* balance globally across regions, DNS- and edge-based, and therefore also absorb volumetric attacks close to their source. The laboratory builds the idea locally with one Nginx.

Because instances appear and disappear with demand, a fixed list of servers cannot be secured; only a *pattern* can. Session 5 turns this into hardening the image rather than the machine, and Session 12 into automation.

3.4 The minimum viable network

A real service is a set of *tiers*: a web tier that speaks HTTP to the world, an application tier for business logic, and a database tier for durable data, with the load balancer in front. Nextcloud follows this pattern on a small scale: web server, PHP application code and a database, which from Session 5 all sit on the students' single VM.

The CSA Security Guidance calls the secured form of this arrangement the *minimum viable network* (MVN) (CSA, 2024, pp. 154–155). An MVN contains no more network than the application needs, so each tier accepts connections only from the tier directly above it, and only on the ports that tier uses. The Internet reaches only the load balancer, on HTTPS; the web tier answers only the balancer; the application tier only the web tier; the database only the application tier; and everything not explicitly permitted is dropped. An attacker who wants the database therefore has no path to it except through the application. Two ideas carry forward from this: *default-deny*, which Session 4 turns into firewall rules, and layering, which Session 4 calls defence in depth. In both, the legitimate flows are listed and the rest is forbidden; attackers are never listed.

3.5 Availability as a number: the SLA

Providers commit to availability by contract.

Definition 3.3 (Service-level agreement)**.** A **service-level agreement** (SLA) describes the level of service a provider commits to, typically as a percentage of uptime over a period.

Two design rules follow. First, deploying across availability zones (or availability sets) removes single points of failure and thereby raises the promised uptime. Secondly, chaining several services multiplies their availabilities, so a solution built from many components has

a lower combined SLA than any single component. More components therefore mean less availability unless redundancy is added on purpose.

Example 3.2. A service chains three components: a load balancer at 99.99 %, a virtual machine at 99.9 % and a database at 99.95 %. Every request needs all three, so the combined availability is the product:

$$0.9999 \times 0.999 \times 0.9995 \approx 0.9984 \;=\; 99.84\,\%.$$

The result appears close to the inputs until it is translated into hours: 0.16 % of a year is about fourteen hours of expected downtime, against roughly one hour for the load balancer alone. Every chained component spends availability, and only deliberate redundancy buys it back. Availability is therefore an architectural property that can be quantified and designed for. Session 11 develops this thread.

3.6 Denial of service

Definition 3.4 (DoS and DDoS)**.** A **denial-of-service** (DoS) attack describes an attempt to make a service unavailable to legitimate users by exhausting a resource (bandwidth, connections, CPU or memory). A **distributed** denial-of-service attack (DDoS) describes the case in which the load comes from thousands of compromised machines at once, a *botnet*, which makes the traffic hard to distinguish from real users and hard to block by source.

DoS attacks availability directly, and DDoS attacks come in three types, ordered by how far up the stack they strike. *Volumetric* attacks saturate the network link, for example with UDP floods and amplification, and are measured in gigabits or terabits per second. *Protocol* attacks exhaust connection state in servers or firewalls, the classic case being the SYN flood, and are measured in packets per second. *Application-layer* attacks send cheap-looking but expensive requests that exhaust the application itself, for example a flood against a search endpoint, and are hard to tell apart from real users. The higher up the stack the attack strikes, the smaller it needs to be and the more it resembles legitimate traffic, and the Nginx access log is where that difference shows.

Example 3.3 (Continuation of Example 1.4)**.** The vocabulary of Definition 1.3 applies to the new threat. *Threat*: an application-layer flood against the service's most expensive endpoint. *Risk*: the likelihood is moderate to high, because such floods need no large botnet and little skill, and the impact is high, because Example 3.1 showed how quickly a single machine reaches the wall. *Control*: not one but several layers, the subject of the next section. In one sentence: availability was at stake, on an IaaS machine the defence is the customer's responsibility, and the single most effective first control is a rate limit.

3.7 Defending availability in layers

Defence against engineered load is layered, and the layers split by who operates them. On the customer's own server, *rate limiting* caps requests per client, in Nginx through the `limit_req` directive, and thereby blunts application-layer floods. *Timeouts and connection limits* stop slow or stuck clients from hoarding state. *Caching* serves repeated content cheaply and so raises the effective capacity of the expensive paths. At cloud scale, *autoscaling* adds capacity under load, *CDNs and anycast* absorb volume close to its source, *cloud DDoS protection* scrubs traffic upstream, and the *WAF* filters malicious requests at Layer 7 (Session 8). The attack types and the defences pair up: volume is absorbed at the edge, protocol floods are the platform's business, and the application layer is where the customer's own configuration (rate limits, caching, well-designed endpoints) carries the weight. Rate limiting is therefore the first hands-on control of the semester: small, local and effective.

3.8 Laboratory: Apache2 and Nginx under load

Exercise 3 installs a web server, loads it and reads the evidence. The packages are installed with `sudo apt install nginx apache2-utils`, and the welcome page is checked in a browser. Two names appear here. `apache2` (the Apache HTTP Server) and `nginx` are the two dominant web servers, and they answer the concurrency problem of Definition 3.2 differently. Apache traditionally dedicates a process or thread per connection, which is simple but heavier under many connections. Nginx uses an event-driven model that handles thousands of connections in a few processes. The course serves with Nginx and uses Apache as the second server in the load-balancing exercise. The load-testing tool `ab` ships with Apache's utilities, which is why `apache2-utils` is installed even to test Nginx. The procedure has four steps. Open `htop` or `btop` in one terminal. Run `ab -n 2000 -c 100 http://localhost/` in another and watch CPU climb while `ab` reports its requests-per-second figure. Tail `/var/log/nginx/access.log` and match the spike in the log with the spike in `htop`, the same event seen by two tools. Optionally add a `limit_req` rule, repeat the run, and compare the failed-request count. All output is saved and labelled.

A load-testing tool and a denial-of-service tool are the same program; the difference is authorisation. Pointed at `localhost`, `ab` is measurement and homework. Pointed at a machine one neither owns nor has written permission to test, the same command is an attack, a criminal offence in Norway as in essentially every jurisdiction, and a disciplinary matter at the university, regardless of intent or outcome. In this laboratory, everything attacked belongs to the attacker.

Reading for this chapter. Comer, §1.5 (From Clusters to Web Sites and Load Balancing, pp. 8–9), Chapter 2 (Elastic Computing and Its Advantages, pp. 15–22) and Chapter 14, §§14.2–14.7 (client–server, scaling, and stateless servers, pp. 195–200); CSA Security Guidance, Domain 7, Section 7.2 (Cloud Network Fundamentals, including the minimum viable network, pp. 153–162). Optional deepening: Jøsang, Chapter 2 (Attack Vectors and Malware, pp. 25–42, on DoS/DDoS). The Microsoft Learn module is *Describe Azure compute and networking services*; students map the session's ideas onto Load Balancer, Application Gateway and Azure DDoS Protection.

3.9 Summary

A web server answers HTTP requests with finite capacity, because concurrency, workers and a cost per request set a ceiling (Definitions 3.1 and 3.2). Load is measured with `ab`, `htop` and the access log, and every service has a wall (Example 3.1). Availability fails when demand exceeds capacity, and a DDoS attack engineers exactly that, in volumetric, protocol or application-layer form (Definition 3.4). The defence is therefore layered: rate limiting, timeouts and caching on the customer's server; autoscaling, CDNs, scrubbing and the WAF in the cloud. Availability can also be counted: SLAs promise it, zones raise it, and chained components multiply it down (Example 3.2). Session 4 decides which traffic may reach the service at all.

Self-check

Sketch answers follow the exercises.

Exercise 3.1 (The load–security bridge)**.** State the bridge sentence connecting load to security, and explain why capacity must be measured rather than estimated.

Exercise 3.2 (Reading an ApacheBench command)**.** Explain the command `ab -n 2000 -c 100 http://localhost/`: what exactly happens, and which three numbers in its report matter most?

Exercise 3.3 (The three DDoS types)**.** Name the three DDoS types, one example each, and state which is hardest to distinguish from legitimate traffic, and why.

Exercise 3.4 (Computing a composite SLA)**.** A solution chains two services at 99.95 % and one at 99.5 %. Compute the combined availability and the expected yearly downtime (1 year $\approx$ 8760 h).

Exercise 3.5 (An application-layer flood)**.** An attacker floods a search endpoint with plausible-looking queries. Which defence layer carries the weight, which concrete Nginx directive implements it, and why would a CDN alone not suffice?

Sketch answers. (1) A denial-of-service attack is demand engineered to exceed capacity; capacity depends on hardware, configuration and content, so only a controlled load test reveals where a specific service's wall stands. (2) Send 2000 requests, 100 concurrently, to the local server; observe requests per second, time per request, and the share of failed requests. (3) Volumetric (UDP flood/amplification), protocol (SYN flood), application-layer (flooding a search endpoint); the application layer is hardest to tell apart because each request resembles a real user, so only patterns in the logs reveal it. (4) $0.9995 \times 0.9995 \times 0.995 \approx 0.9940 = 99.40\,\%$; about $0.60\,\% \times 8760 \approx 53$ hours per year. (5) The application layer: `limit_req` caps requests per client; a CDN absorbs volume but forwards plausible dynamic queries, so the expensive endpoint still burns, and the rate limit (plus caching) must sit where the cost arises.

Chapter 4

Networking and firewall controls

Session 3 measured how much traffic the service survives; this chapter decides which traffic may reach it at all. It covers addressing and ports, the software-defined networks of the cloud, the firewall and its variants, and defence in depth and Zero Trust. In the laboratory the VM of Session 2, now serving Nginx, receives a default-deny policy: SSH only from the administrator, the web port for everyone, nothing else.

4.1 Addresses, ports and the attack surface

Every firewall rule is written in the vocabulary of addressing. Two machines on a network find each other by *IP address*, and on one machine a service is reached at a *port*, a sixteen-bit number that identifies which program the traffic is for. A web server listens on port 80 (HTTP) or 443 (HTTPS), SSH on 22, a database on 5432 or 3306. The pair *(address, port)* therefore names one service on one machine, and the combination *(protocol, address, port)* on both ends, carried over TCP (connection-oriented, ordered) or UDP (connectionless), is what a firewall rule matches. `ss -tulpn` lists one's own side of this: every port the machine listens on.

Definition 4.1 (Attack surface)**.** The **attack surface** of a system describes the sum of the points at which an attacker can try to interact with it: every open port, every reachable service, every exposed endpoint. Reducing the attack surface means removing such points, since a service that is not listening, or a port that is not reachable, cannot be attacked through that route.

Comer (2021) uses the term *security attack surface* in the same sense for microservices and remote access (Comer, 2021, pp. 167–168, 234). Every listening port is a door, some doors must be open for the service to work, and the security question is which. For this reason the first useful action on any machine is to run `ss -tulpn` and to justify each line.

4.2 Networks in the cloud are software

In a data centre the network a VM sits on is defined in software (Comer, 2021, Chapter 7). Because many tenants share one physical fabric, each tenant gets a *virtual network* built as an *overlay* on the shared physical *underlay* (Comer, 2021, pp. 88–89). VLANs, and their data-centre-scale successor VXLAN, tag traffic so that one tenant's packets never mix with another's (Comer, 2021, pp. 89–90). A *virtual switch* inside each physical server connects the VMs running on it (Comer, 2021, p. 91). *Network address translation* (NAT) lets privately addressed machines reach the outside world (Comer, 2021, pp. 91–92).

Boundaries between networks are therefore logical and enforced by configuration, with two consequences: they can be misconfigured, and they can be audited by reading the configuration.

Definition 4.2 (Software-defined networking)**.** **Software-defined networking** (SDN) describes the separation of a network's *control* (the decisions about where traffic goes) from its *forwarding* (the act of moving packets), so that a program, the *controller*, configures and monitors the switches. In the cloud this is why an entire network can be created, changed or destroyed through an API in seconds, as a VM can.

Comer (2021) describes OpenFlow, the first SDN protocol, and a second generation in which switches run small programs of their own (Comer, 2021, pp. 94–96). Because the network is software, security is enforced in the fabric itself, per resource, rather than by a physical appliance the traffic happens to pass (CSA, 2024, p. 154). The tool for that is the security group.

Definition 4.3 (Security group)**.** A **security group** describes a virtual firewall enforced by the cloud network fabric at the level of an instance, network interface or subnet. Its rules allow or deny traffic by IP address, port and protocol. It is *stateful*: if a rule permits an outgoing request, the corresponding reply is allowed back automatically.

Two details matter in practice. First, security groups are stateful and attach to resources, whereas *network access control lists* (NACLs) work lower in the stack, are typically stateless and apply to subnets (CSA, 2024, p. 156); the distinction decides which tool fits which job. Secondly, providers differ in their defaults. AWS starts from a deny-all position, so the customer must add allow rules, and two resources in the same security group cannot talk to each other until a rule permits it (CSA, 2024, p. 158). Azure, in contrast, starts from a more permissive position: the built-in default rules of a network security group allow traffic freely within the virtual network (and from the Azure load balancer), while a final default rule denies inbound traffic from the Internet. The Internet is thus closed by default, but east–west traffic between the customer's own resources is open unless restricted, and that is exactly the lateral movement of Session 2. A provider's defaults must therefore be known before they are trusted, because assuming the wrong one leaves a service reachable from an unintended place.

4.3 The firewall

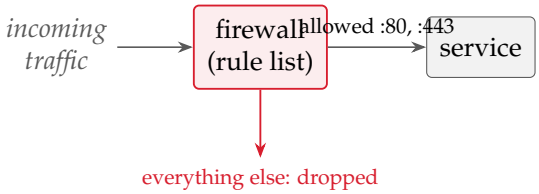

Figure 4.1. A firewall under default-deny. Only traffic an explicit rule permits (here, the web ports) reaches the service; the final catch-all rule drops the rest, so anything unforeseen fails closed.

Definition 4.4 (Firewall, default-deny)**.** A **firewall** describes a component that inspects network traffic and permits or blocks it against a set of rules. Under a **default-deny** (allow-list) policy the final, catch-all rule blocks everything, so only traffic that an earlier rule explicitly permits gets through. The opposite, default-allow (block-list), permits everything not explicitly forbidden and fails *open*: anything not foreseen is allowed.

Default-deny is the stance of the minimum viable network of Session 3: list the legitimate flows and forbid the rest. A further distinction is *stateless* versus *stateful*. A stateless firewall judges each packet alone, whereas a stateful one remembers active connections, so that having allowed a connection outward, it lets the replies return without a separate rule. Security groups and the host firewall of the laboratory are stateful.

The host firewall on Ubuntu is `ufw` (uncomplicated firewall), a front end to the kernel's packet filter, and its opening sequence encodes default-deny directly:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw enable
```

The sequence refuses all incoming traffic by default, permits outgoing traffic, adds two exceptions (SSH and HTTP) and switches the policy on. Afterwards `sudo ufw status` prints the loaded policy, and it is always read back, because the gap between the rules one believes one wrote and the rules that are loaded is where incidents hide. Three mistakes are common. The first is leaving SSH open to the whole Internet rather than to the administrative address, the risk of Example 1.4. The second is opening a database port for testing and forgetting it. The third is assuming closed when the default is open, as on Azure.

Example 4.1 (Example 1.4, implemented)**.** Session 1 named a threat, judged its risk and prescribed a control for the open SSH port; here the control is built. *Threat*: automated scanners brute-force the SSH login within minutes of the port appearing. *Risk*: likelihood high (constant scanning), impact high (a login is full control). *Control*: allow SSH only from the administrator's own address, with `sudo ufw allow from YOUR_IP to any port 22 proto tcp` instead of the blanket `allow 22/tcp` above, and keep the web port open to all. One `ufw status` now shows which CIA property each rule protects and whose responsibility the rule was.

4.4 Firewall types, NGFW and WAF

A plain firewall (`ufw`, a security group, an Azure NSG) judges traffic by address, port and protocol, and it cannot see inside an allowed connection: once port 443 is permitted, everything on port 443 passes, malicious payloads included. The firewall family is therefore ordered by how deeply each member looks. The *packet-filter* firewall judges each packet in isolation by address, port and protocol: stateless, fast, and blind to context. The *stateful* firewall adds memory of active connections, and `ufw` and every cloud security group are of this kind. Both stop at Layer 4, so they know where traffic goes but not what it carries. The *next-generation firewall* (NGFW) looks inside: it performs *deep packet inspection* (Comer, 2021, p. 235) to identify the application in a flow, applies rules by user identity rather than only by address, integrates *intrusion prevention* (Session 9), and consults threat-intelligence feeds of known-bad addresses and domains. Palo Alto, FortiGate and Cisco Firepower are on-premises examples; Azure Firewall Premium and AWS Network Firewall are cloud examples.

In the cloud the choice is one of deployment (CSA, 2024, p. 159). A *CSP firewall* such as Azure Firewall or AWS Network Firewall is built into the platform, so there is nothing to maintain, but it is limited in customisation and advanced features. A *virtual appliance*, a vendor NGFW or IDS/IPS run as a VM, offers more control at the cost of patching and scaling it oneself. The limit of the managed firewall is concrete: Azure Firewall's application rules filter by domain name only, without TLS termination or deep packet inspection, and the vendor documentation itself calls them very limited compared with a next-generation firewall such as a Palo Alto appliance.

The *web application firewall* (WAF) is not a network firewall and must not be confused with one. It sits in front of a single web application and inspects HTTP requests at Layer 7 for application-layer attacks (SQL injection, cross-site scripting, the application-layer floods of Session 3), matched against rule sets such as the OWASP core rules (CSA, 2024, p. 159). An NGFW thus secures the network broadly, whereas a WAF secures one web application specifically. Table 4.1 compares them.

Figure 4.2 shows the same comparison as a grid of layers; a filled cell means the tool operates there.

Tool	Layer	Inspects	Example
Packet-filter firewall	L3–L4	address, port, protocol; each packet alone (stateless)	early `iptables` rule
Stateful firewall	L3–L4	the above plus connection state (replies allowed back)	`ufw`, cloud security group, Azure NSG
Next-gen firewall (NGFW)	L3–L7	application identity, payload (DPI), user, threat intelligence; IPS built in	Palo Alto, Azure Firewall Premium
Web app firewall (WAF)	L7 (HTTP)	HTTP request content of one web application (SQLi, XSS, floods)	Azure App Gateway WAF, ModSecurity

Table 4.1. Firewalls and the WAF, ordered by inspection depth. A WAF is not a better firewall: it guards one application at Layer 7, where a network firewall does not operate.

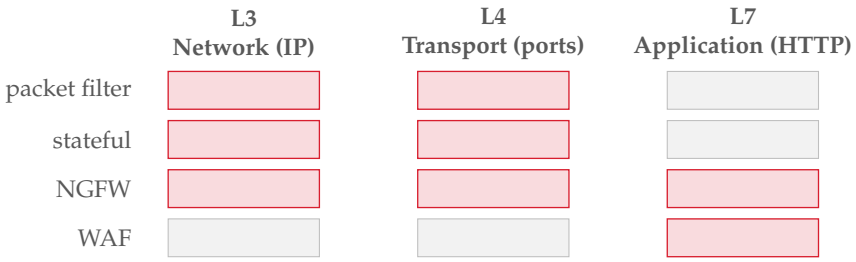

Figure 4.2. Which layers each tool operates at (filled = yes). Packet-filter and stateful firewalls act at Layers 3–4 (addresses, ports); an NGFW reaches up through Layer 7 with deep packet inspection; a WAF operates only at Layer 7, on the content of one web application.

One idea links the WAF to the intrusion prevention of Session 9. A WAF runs either in *detection* mode, where it logs matching requests and lets them through, or in *prevention* mode, where it blocks them, and that is the same choice that separates an IDS from an IPS. A WAF prevents attacks from reaching an application, but it cannot repair a flaw that is already there, and it is no substitute for writing the application properly (CSA, 2024, p. 247). Session 8 covers the WAF in full; on the students' service it runs in front of Nginx.

Real-world case: Capital One, 2019

A chain of misconfigurations exposed about 106 million credit-card applications. The entry point was a misconfigured web application firewall running on a cloud instance whose attached identity held far more permission than it needed. A server-side request forgery flaw made the WAF query the cloud platform's internal metadata service, which returned the temporary credentials of that identity. The credentials could list and copy the contents of more than 700 storage buckets. Three facts matter for this course. A security control can itself become the attack vector. The damage was set not by the WAF but by the excessive privilege of the identity behind it, a least-privilege failure (Session 7). And standard monitoring did not flag the theft, because it looked like ordinary authorised API calls (Session 9).

4.5 Defence in depth and Zero Trust

No single control is sufficient, so controls are layered. *Defence in depth* means independent layers, so that the failure of one does not expose everything behind it: network segmentation on the outside, host firewalls next, then application controls, identities, and encrypted data at the centre. The tiered minimum viable network of Session 3 is one instance. A *bastion host*, a

single hardened and closely watched machine that is the only entry point for administration, is another, and segmenting the network into tiers that may speak only to their neighbours is a third.

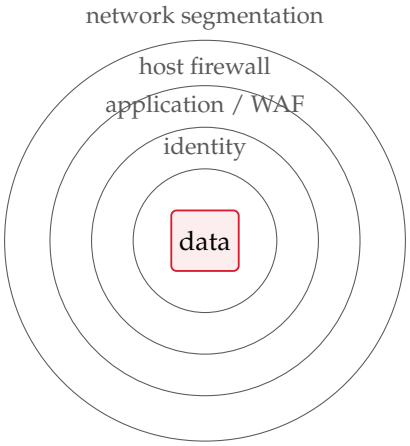

Figure 4.3. Defence in depth. Independent layers around the asset, so that the failure of any one does not expose everything behind it. Each later session adds a ring.

Definition 4.5 (Zero Trust)**. Zero Trust** describes a security strategy in which nobody and nothing is trusted merely because of where it sits on the network. It assumes that a breach has happened or will happen, and it therefore checks every user, device, application and transaction again and again, no matter whether a request comes from inside the network or from outside (CSA, 2024, p. 168).

The Guidance contrasts Zero Trust with the traditional castle-and-moat perimeter and regards the latter as no longer effective now that services live in the cloud and staff work remotely (CSA, 2024, p. 50). In the old model a hard perimeter guarded a trusting interior, so an attacker who breached the wall could move freely. Comer (2021) gives the cloud reason for abandoning it (Comer, 2021, pp. 235–237). Traditional security rested on a firm split between insiders and outsiders, enforced at the perimeter by firewalls, deep packet inspection and a *demilitarised zone* that exposed only a few chosen servers. In the cloud, however, the provider's placement algorithms run many tenants' virtual machines on the same physical hardware, and even with separate virtual networks their traffic crosses the same physical links. A tenant therefore has no physical perimeter and cannot sort the world into inside and outside. Privileges must instead be assigned per service and every request validated separately, using the identity of the user and the device, and nobody is trusted for having logged in once.

In the NIST reference model (SP 800-207) a *policy decision point* decides and a *policy enforcement point* enforces, both placed in the path of every access (CSA, 2024, p. 171). Supporting technologies include the software-defined perimeter, which hides resources from the network until a client has been authenticated (CSA, 2024, p. 172), and, at the network edge, SASE (CSA, 2024, p. 174). Students are not asked to deploy this stack, only to recognise its principles: least privilege, verify explicitly, assume breach.

Example 4.2 (The red thread, one week early)**.** The VM this week runs one exposed service (Nginx) and one administrative door (SSH), and the firewall closes everything else. In Session 5 Nextcloud arrives as a set of tiers (web server, application, database), and the default-deny stance scales up to the minimum viable network of Session 3. Each tier then accepts traffic only from the tier above it, and the database is reachable from nowhere but the application. No new principles are needed for that, only more rules. Defence in depth is therefore not a product to buy, but default-deny applied at every layer one owns.

Reading for this chapter. Comer, Chapter 7 (Virtual Networks: overlays, VLANs, VXLAN, NAT, SDN and OpenFlow, pp. 87–96) and Chapter 17, §§17.3–17.5 (traditional perimeter security, why it fails in the cloud, and the Zero Trust model, pp. 235–237); CSA Security Guidance, Domain 7, Section 7.2 (Cloud Network Fundamentals, including security groups and NACLs, pp. 153–162) and Section 7.4 (Zero Trust & Secure Access Service Edge, pp. 168–176). Optional deepening: Jøsang, Chapter 6 (Network Security, incl. firewalls in §6.4, pp. 117–142). The Microsoft Learn module is *Describe Azure networking services*; students map the session's ideas onto Virtual Networks, Network Security Groups and Azure Firewall.

4.6 Summary

Machines address one another by IP and reach services by port. The pair, with the protocol, is what a firewall rule matches, and every listening port is part of the attack surface (Definition 4.1). In the cloud the network is software: overlays on a shared underlay, programmable through SDN (Definition 4.2) and secured in the fabric by stateful security groups (Definition 4.3), whose defaults differ by provider and are never assumed. The firewall (Definition 4.4) runs default-deny, and in the laboratory a handful of `ufw` rules close the SSH door of Chapter 1 (Example 4.1). Plain filters cannot see inside allowed traffic, so an NGFW adds application awareness and deep packet inspection, and a WAF guards one web application. Defence in depth layers these controls, and Zero Trust (Definition 4.5) removes the trusted interior. Session 5 installs Nextcloud on the machine.

Self-check

Sketch answers follow the exercises.

Exercise 4.1 (Attack surface)**.** Explain the term attack surface in one's own words, and state why `ss -tulpn` is a security command and not merely an administrator's one.

Exercise 4.2 (Default-deny reasoning)**.** State the default-deny principle, and explain why it is the same reasoning as the minimum viable network of Session 3.

Exercise 4.3 (A cloud firewall default)**.** A team member says: "I added an allow rule on Azure, so now only the web port is open." Why might the service still be exposed, and what does the answer have to do with AWS behaving differently?

Exercise 4.4 (NGFW vs WAF)**.** Distinguish an NGFW from a WAF in one sentence each, and state which one applies against an application-layer flood on a Nextcloud login page.

Exercise 4.5 (SSH brute force as threat–risk–control)**.** Write, in the vocabulary of Definition 1.3, the threat, risk and control for an SSH port left open to the whole Internet, and give the `ufw` rule that implements the control.

Sketch answers. (1) The attack surface is every point an attacker can interact with: open ports, reachable services, exposed endpoints; `ss -tulpn` enumerates the listening ports, so it measures that surface and allows each door to be justified. (2) Default-deny blocks everything except explicitly permitted traffic; like the MVN, it works by listing the few legitimate flows and forbidding the rest, which needs no knowledge of the attacker. (3) Azure's network security groups allow east–west traffic within the virtual network by default (while denying inbound from the Internet), so intra-VNet paths may be open even when no rule was added; on AWS a security group denies all inbound traffic, including between resources in the same group, until it is allowed. The mental model is the same, namely know the default before trusting it, and assuming the wrong one can leave a resource reachable from an unintended location.

(4) An NGFW secures the network broadly with application awareness and deep packet inspection; a WAF secures one web application by inspecting HTTP requests, so against an application-layer flood on the login page the WAF applies (Session 8). (5) Threat: automated brute-forcing of the SSH login. Risk: high likelihood (constant scanning), high impact (full control on success). Control: restrict SSH to the administrative address with `sudo ufw allow from YOUR_IP to any port 22 proto tcp`.

Chapter 5

Cloud-native versus lift-and-shift

This chapter installs Nextcloud as a container on the hardened VM. The choice of a container raises the central question of the week: an application can be carried into the cloud unchanged, or it can be rebuilt the way the cloud works, and the difference is a security difference. The chapter therefore covers the two migration paths, containers, the kernel beneath them, and the immutable workload model.

5.1 Two ways into the cloud

Comer (2021) separates *conventional* software, written for a fixed machine that an administrator tends, from *cloud-native* software, written to run as many interchangeable instances that the cloud creates and destroys as needed (Comer, 2021, p. 146). The CSA Security Guidance turns the same distinction into migration strategies (CSA, 2024, pp. 152–153).

Definition 5.1 (Lift-and-shift and cloud-native)**. Lift-and-shift** (rehosting) describes moving an application to the cloud with as little change as possible: the same server, operating system and management, now on a rented VM instead of an owned one. **Cloud-native** (re-architecting or rebuilding) describes redesigning the application to use the cloud's own patterns (interchangeable instances, containers, managed services, automated scaling) so that it can be created, replaced and scaled by software.

The Guidance names three strategies on this spectrum (CSA, 2024, pp. 152–153). Rehosting is lift-and-shift with the least change. Refactoring adjusts the application to use some cloud services. Re-architecting or rebuilding is fully cloud-native: the most change and the most benefit, the highest cost in time and people, and the only path on which security can be designed in from the start. Lift-and-shift is fast and low-risk to perform, because nothing changes and therefore nothing breaks. Cloud-native is slow and demanding, because it needs new skills, new tools and rewriting. The rest of this chapter gives the security reason for resisting the faster path where possible.

5.2 Containers

Definition 5.2 (Container)**.** A **container** describes a package of an application together with its libraries and dependencies in a single image that runs as an isolated process on a host, sharing the host's operating-system kernel rather than carrying its own. It is lightweight (megabytes rather than gigabytes), starts in seconds and runs identically wherever the container runtime runs.

A virtual machine virtualises the hardware, so a whole guest operating system runs on emulated devices. A container instead virtualises the operating system: the host kernel uses

isolation features (Linux *namespaces*) to give each container its own view of processes, files and network, while all containers share one kernel (Comer, 2021, pp. 73–77). The dominant technology is Docker. A plain-text *Dockerfile* names a base image, the software to add and the command to run; a build step turns the recipe into an image; and the image runs as a container anywhere (Comer, 2021, pp. 81–83). The same image therefore runs the same way on a laptop, in the laboratory and in the data centre, which is what lets the cloud treat instances as interchangeable.

Because a container shares the host kernel, the wall between two containers is thinner than the wall between two virtual machines. A VM is a whole computer in software, whereas a container is a packaged process. Cloud-native systems use containers because packaged processes are cheap to start, stop and replicate, and VMs remain the stronger isolation boundary beneath them.

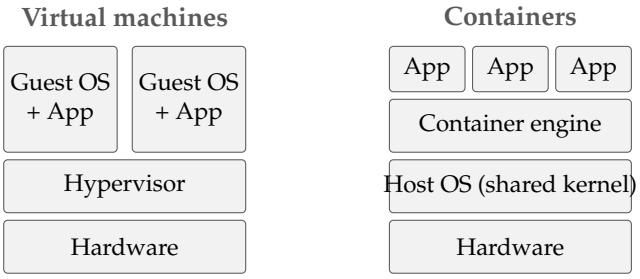

Figure 5.1. A VM carries a whole guest operating system on virtualised hardware; a container is a packaged process sharing the host kernel: lighter, but with a thinner isolation wall.

5.3 The kernel

The *kernel* is the single program that runs in the processor's privileged mode (Section 2.4). It owns the hardware, schedules every process, manages memory and is the only gateway through which a program reaches a resource; a program asks by making a *system call*, which traps into the kernel. Everything else (the shell, Nginx, Nextcloud) is *user space*, and the set of system calls is the contract every program on that system is compiled against.

That contract differs between the three operating-system families. Linux runs a *monolithic* kernel: scheduler, memory manager, filesystems, network stack and drivers form one privileged program in one address space, extended at run time by loadable modules. The Linux kernel also provides the two mechanisms containers are built from. *Namespaces* give a process an isolated view of one class of resource, such as its own process list, network interfaces, filesystem root or user IDs (Comer, 2021, p. 74), and *control groups* (cgroups) cap how much CPU, memory and I/O it may consume. A container is therefore an ordinary Linux process wrapped in a set of namespaces and a cgroup; there is no machine inside it.

Windows runs the *NT* kernel, a *hybrid* design whose kernel-mode Executive sits above a small core, and its system calls and its ideas of processes, objects and security are its own. Windows containers accordingly use Windows kernel features and run Windows images. macOS runs *XNU*, a hybrid of the *Mach* microkernel and a *BSD* (UNIX) layer with a driver framework on top. It presents a UNIX/POSIX interface, which is why its shell resembles Linux's, but its kernel has no built-in idea of Linux containers.

A Linux container therefore needs a Linux kernel, because the image holds libraries and binaries but no kernel. `docker run` on a Windows or macOS laptop consequently does not run Linux containers on Windows or Darwin. Docker Desktop instead runs a small Linux virtual machine in the background, on Windows through WSL 2 and on macOS through the platform's virtualisation framework, and the containers run on that Linux kernel. Containers sit on top of virtualisation; they do not replace it (Comer, 2021, pp. 71–75).

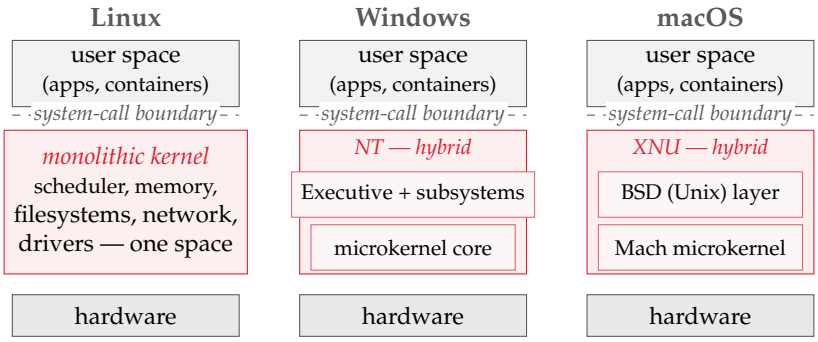

Figure 5.2. Three kernels, one shape. All three enforce the same user/kernel split, reached only through system calls, but differ inside: Linux is *monolithic* (one privileged program), whereas Windows (NT) and macOS (XNU) are *hybrid*, layering an outer service layer over a small core. A Linux container's isolation is enforced by the Linux kernel, which is why it requires one.

The security point follows. The wall around a virtual machine is the hypervisor's hardware-enforced boundary, whereas the wall around a container is the kernel itself, enforcing namespace separation between processes on one kernel. A flaw in the shared kernel is therefore shared by every container on the host, and a *container escape*, code breaking out of its namespaces into the host kernel, endangers every other container and the host at once, whereas a VM escape must first defeat the hypervisor. For this reason the host kernel belongs to each container's trust boundary and must be kept patched, and a workload that needs a hard boundary still gets its own VM.

5.4 Why containers, and orchestration

A virtual machine carries a whole guest operating system, which takes minutes to boot and gigabytes of memory (Comer, 2021, p. 72). That is acceptable for a long-running server but too heavy when a service must add and drop instances by the second to follow demand, and a container starts within that second and costs megabytes. The need to scale on demand is therefore the reason why containers rather than VMs became the unit of cloud-native deployment; the thinner wall is the price.

A service made of many short-lived containers on many machines needs something that decides where each one runs, restarts the ones that die and scales their number. That task is *orchestration*, and Comer (2021) covers the dominant system, Kubernetes (Comer, 2021, pp. 127–140). Kubernetes groups containers into *pods*, schedules them onto worker nodes and runs a *control plane* that continuously drives the running system toward a declared desired state. Students do not operate Kubernetes, but two of its ideas return. Its control plane is a management plane and therefore a prime target (Session 7), and its continuous comparison of reality with a declared state is the control-loop thinking behind monitoring (Session 9) and declared infrastructure (Session 8).

5.5 Ephemeral versus immutable workloads

The CSA Security Guidance describes two ways of treating a workload over its life, and here the architectural choice becomes a security choice (CSA, 2024, pp. 179–180).

Definition 5.3 (Ephemeral and immutable workloads)**.** A workload is **ephemeral** (short-running) when its instances are treated as disposable: created from a fixed image to do their work and then discarded. It is **immutable** when a running instance is never modified in place, so that to change or fix it a new image is built and the instance replaced rather than patched. The opposite,

mutable long-running model keeps instances alive for long periods and maintains them by hand.

One note on terms is necessary. The heading of the relevant Guidance section attaches the label immutable to the long-running model, whereas the text beneath it describes immutable infrastructure as the replace-rather-than- patch practice of the short-running model (CSA, 2024, p. 179). This script follows the text, which matches industry usage.

Cloud-native architectures run mostly ephemeral, immutable workloads: security is built into the image, and a compromised or outdated instance is replaced rather than repaired. Lift-and-shift, in contrast, produces long-running instances carried over from the on-premises world, maintained by hand and patched in place. The Guidance regards short-running workloads as the safer model for three reasons (CSA, 2024, p. 180). An instance that lives for minutes gives an attacker little time. Every instance is built by automation from the same tested image, so no two drift apart. And one image can be tested once, whereas a long-running server must be assessed again and again. Immutable infrastructure thus prevents the slow accumulation of unpatched, drifted, slightly different servers.

This changes a lesson of Session 2. Patching an IaaS machine with `apt` remains correct for a lift-and-shift VM. In the cloud-native world, however, the running instance is not patched at all; the image is rebuilt with the fix and fresh instances are rolled out. Both approaches are legitimate, and recognising which world applies tells the administrator which move is correct.

Example 5.1 (One vulnerability, two worlds)**.** A critical vulnerability is published for a library Nextcloud uses. In the lift-and-shift world, with Nextcloud on a long-running VM, the control is the one from Chapter 1: patch promptly, in place, on that machine, with the residual risk of a second copy elsewhere. In the cloud-native world, with Nextcloud as a container built from an image, the control is to update the image, rebuild, and replace every running container with the new version, retiring the vulnerable ones entirely. The threat and the CIA properties at stake (integrity and confidentiality of the files) are the same. The controls differ because the two worlds treat a running instance differently: one as an asset to be maintained, the other as a part to be swapped.

5.6 Securing workloads in a moving environment

When instances appear and vanish with demand, security practices inherited from static data centres stop fitting, and the Guidance names the adjustments (CSA, 2024, pp. 180–181). Monitoring agents must be small, must understand the cloud, and must not depend on a fixed IP address or any other setting that stays the same, because in an autoscaling group addresses and names are handed out and taken back all the time. For the same reason a new instance registers itself with the monitoring system when it starts, and its log entries carry the identity of the workload rather than only an address. Vulnerability scanning over the network, the traditional method, often no longer works either, because internal networks are closed by default and each workload's security group admits only the connections it needs, so a scanner on the same subnet may be unable to reach its targets. The Guidance therefore offers three alternatives: check each VM or container image at build time, before it is deployed, so that only images known to be good ever run; assess snapshots of running VMs offline; or build assessment agents into the images. The first is the shift-left approach that Session 8 develops into DevSecOps.

The thinner wall has its own controls. Because containers share a kernel, a flaw in the kernel or the runtime can let code escape one container into the host or a neighbour, the container version of the VM escape of Session 2. The Guidance therefore recommends minimal images, so that a container holds less to exploit; least-privilege configuration, including not running containers as root; and a patched runtime.

5.7 The red thread: Nextcloud arrives

Exercise 5 runs Nextcloud as a Docker container on the prepared VM: one command pulls the image, one runs it, and the file-sharing platform is live behind the firewall of Session 4. Nextcloud-in-a-container is an image that can be rebuilt and replaced, so it is the immutable pattern at laboratory scale. The service still answers plain, unencrypted HTTP, however, so every file and password crosses the network unencrypted, and Session 6 closes that gap with TLS.

Reading for this chapter. Comer, Chapter 5 (Virtual Machines, pp. 55–68), Chapter 6 (Containers: §6.3 elasticity on demand, namespaces, Docker and Dockerfiles, pp. 71–83), Chapter 10 (Orchestration and Kubernetes, pp. 127–140) and §11.3 (Cloud-Native vs. Conventional Software, p. 146); CSA Security Guidance, Domain 8, Section 8.1 (Cloud Workload Security: short- and long-running workloads, impact on traditional controls, SCA and SBOM, pp. 177–182) and Domain 7, §7.1.5 (Cloud Migration Architecture: rehost, refactor, re-architect, pp. 152–153). Optional deepening: Jøsang, Chapter 11 (Secure by Design, pp. 243–270). The Microsoft Learn module is *Describe Azure compute* (containers and container instances).

5.8 Summary

Lift-and-shift rehosts an application unchanged, whereas cloud-native rebuilds it around the cloud's own patterns (Definition 5.1), with refactoring in between. The container (Definition 5.2) is a process sharing the host kernel, reproducible and cheap to replicate, but with thinner isolation than a VM, because the shared kernel is part of every container's trust boundary. Cloud-native workloads are ephemeral and immutable (Definition 5.3): security is built into a tested image and instances are replaced rather than patched, which limits exposure, enforces consistency and eases verification, whereas lift-and-shift inherits the long-running, hand-patched model. The same vulnerability therefore meets different controls in the two worlds (Example 5.1), and workload security in a moving environment means cloud-aware agents, assessment of images before deployment and defence of the container wall. Nextcloud is now running as a container on plain HTTP, and Session 6 encrypts the transport.

Self-check

Sketch answers follow the exercises.

Exercise 5.1 (Lift-and-shift vs cloud-native)**.** Explain the difference between a lift-and-shift migration and a cloud-native application, and place refactoring on the spectrum between them.

Exercise 5.2 (Container vs VM isolation)**.** A container and a virtual machine both isolate workloads. What does each virtualise, and which gives the stronger isolation boundary? Name one security consequence of the difference.

Exercise 5.3 (Why immutable workloads are safer)**.** Why does the CSA Guidance consider short-running, immutable workloads more secure than long-running ones? Give the three reasons.

Exercise 5.4 (A vulnerable dependency)**.** A library used by the service has a critical vulnerability. Describe the control in the lift-and-shift world and the control in the cloud-native world, and state why they differ.

Exercise 5.5 (Why network scanning fails)**.** Why does network-based vulnerability scanning largely fail against cloud-native workloads, and what does the Guidance recommend instead?

Sketch answers. (1) Lift-and-shift (rehosting) moves the application unchanged onto a rented VM; cloud-native rebuilds it around interchangeable instances, containers and managed services; refactoring sits between, with the application adjusted to use some cloud services without a full rebuild. (2) A VM virtualises the hardware (its own guest OS on emulated devices); a container virtualises the operating system (its own process, file and network view, sharing the host kernel). The VM's wall is stronger; the container's thinner wall means a kernel or runtime flaw could let code escape between containers, so minimal, least-privilege containers matter. (3) Limited exposure (short-lived instances give an attacker little time), consistency (every instance comes from the same tested image, preventing configuration drift), and easier verification (one image is tested rather than many hand-maintained machines audited). (4) Lift-and-shift: patch the running VM in place, as in Chapter 1. Cloud-native: rebuild the image with the fix and replace the running instances, retiring the vulnerable ones. They differ because the two worlds treat a running instance differently, one maintained and one immutable and disposable. (5) Default-deny security groups block the scanner and there is no stable host or address to scan; instead, the container or VM image is assessed at build time, before deployment, so that only known-good images run.

Chapter 6

TLS and data in transit

Since Session 5, Nextcloud serves every file and every password over plain HTTP, readable by anyone on the network path. This chapter closes that gap. It covers the three states of data, the two families of cryptography, and TLS, which combines them to protect a connection. The laboratory turns `http://` into `https://`.

6.1 Three states of data

Data is always in one of three states, and each is exposed differently, so each needs its own control (CSA, 2024, pp. 210–211).

Definition 6.1 (The three data states)**. Data at rest** describes data held in storage: on a virtual disk, in object storage, in a database. **Data in motion** (in transit) describes data being transmitted between locations, across a network or the Internet. **Data in use** describes data currently being processed in memory by an application. Each state requires its own security measures, and protecting one does not protect the others.

The three states map onto the course. Data at rest is the subject of Session 11 (backup and storage), where disk and object encryption guard against a stolen drive or an exposed bucket. Data in use is guarded mainly by the access controls of Session 7, because the program working on the data must be able to read it, which makes this the hardest state to protect. This session covers the middle state, data in motion, because it is the one the running Nextcloud currently ignores. The Guidance's advice for data in motion is encryption protocols and secure channels that protect confidentiality and integrity in transit (CSA, 2024, p. 211), and TLS is that channel.

6.2 Two families of cryptography

Encryption turns readable *plaintext* into unreadable *ciphertext* under the control of a *key*, reversibly for whoever holds the right key and not otherwise. Tokenisation is a different mechanism: it replaces a sensitive value with an unrelated random token and keeps the mapping in a secure store. There are two families of encryption, and TLS needs both (Jøsang, 2024, pp. 65–80).

Definition 6.2 (Symmetric and asymmetric encryption)**. Symmetric** encryption describes the case in which one shared secret key both encrypts and decrypts. It is fast and suits bulk data, but both parties must already share the key. **Asymmetric** (public-key) encryption describes the case in which each party has a *key pair*, a public key that may be shared freely and a private key kept secret, such that what one key locks only the other unlocks. It solves the key-sharing problem and enables digital signatures, but is far slower than symmetric encryption.

The two families have complementary strengths. Symmetric encryption is fast but cannot deliver the shared key to the other party over an untrusted network without the key being observed. Asymmetric encryption can protect that exchange, because the public key may travel in the open, but it is too slow for a whole video call or file transfer. TLS therefore uses asymmetric encryption once, at the start, to agree on a fresh symmetric key, and then uses that key for the data: slow handshake, fast conversation.

One question remains, and it turns cryptography into a system. A browser that receives a public key claiming to belong to a Nextcloud server needs a reason to believe that the key is the server's and not an impostor's (Jøsang, 2024, pp. 105–112).

Definition 6.3 (Certificate and PKI)**.** A **digital certificate** describes a binding of a public key to an identity (a domain name), signed by a **certificate authority** (CA) that both parties trust. The **public-key infrastructure** (PKI) describes the system of authorities, certificates and trust relationships that makes this work at Internet scale. A browser trusts a server's public key because a CA it already trusts has confirmed, by its signature, that the key belongs to that domain.

Without this binding, encryption alone would be worth little, because a client could hold a perfectly private conversation with an attacker who had placed themselves in between and supplied their own public key. The certificate lets the browser check that it is talking to the right server before it trusts the channel, so authentication here works for confidentiality.

6.3 TLS

Definition 6.4 (TLS)**. Transport Layer Security** (TLS) describes the protocol that secures data in motion for most Internet traffic. In a short *handshake* the parties authenticate the server through its certificate, use asymmetric cryptography to agree on a symmetric session key, and then encrypt the rest of the connection with it. HTTP carried inside TLS is **HTTPS**, the padlock in the browser.

The handshake is the two families of Definition 6.2 working together: the certificate (PKI) proves who the server is, asymmetric encryption establishes a shared secret, and symmetric encryption then protects the bulk traffic. TLS thereby delivers three properties for data in motion: confidentiality, since eavesdroppers see only ciphertext; integrity, since tampering in transit is detected; and server authentication, since the client is talking to the certified server.

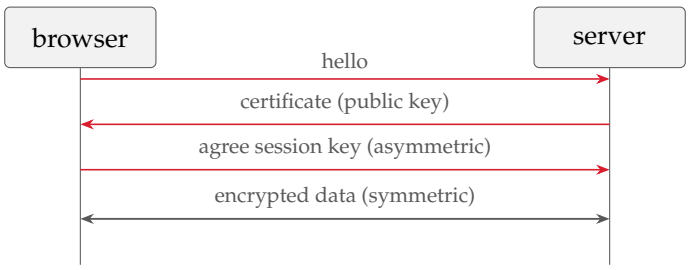

Figure 6.1. The TLS handshake. The certificate authenticates the server and carries its public key; asymmetric cryptography then agrees a fresh symmetric session key, which encrypts the rest of the connection: slow handshake, fast conversation.

Four practical points carry into the laboratory. First, versions matter: TLS 1.2 is the minimum and TLS 1.3 the current version, whereas TLS 1.0 and 1.1 survive only for backward compatibility and are disabled, because a downgrade to a broken version undoes the protection. Secondly, certificates must be obtained and kept valid, because an expired certificate breaks the service as surely as a misconfiguration. Thirdly, in the cloud the private keys behind certificates belong in

a managed key store (Azure Key Vault, AWS KMS) rather than in a file beside the application, and the Guidance advises a customer of IaaS or PaaS to begin with the key management service the provider offers (CSA, 2024, p. 229). Lastly, TLS protects data in motion between endpoints and does nothing for the same data once it sits decrypted on the server (at rest) or is processed there (in use); encrypting the channel is necessary, not sufficient.

Encryption also has a cost for the defenders of Session 4: a firewall or NGFW performing deep packet inspection sees only ciphertext inside a TLS connection. Enterprises sometimes resolve this with *TLS inspection* (TLS termination, break-and-inspect), decrypting traffic at a gateway, examining it and re-encrypting it onward. This restores inspection but breaks end-to-end confidentiality at one trusted point, with the concentration risk that implies. A control that serves one property can thus hinder another, and the engineer decides the trade-off rather than ignoring it.

Example 6.1 (http becomes https)**.** The file of Chapter 1, `salary_list.pdf`, is now served by the Nextcloud deployed in Session 5. *Threat*: an attacker on the network path (shared Wi-Fi, a compromised router) reads the file and the login password as they cross in plain HTTP. *Risk*: on any untrusted network the likelihood is real and the impact high, because it covers the confidentiality of the file and of credentials that unlock far more. *Control*: serve the site over HTTPS, so that the certificate authenticates the server and TLS encrypts the transport; the interceptor now sees only ciphertext. This defends the confidentiality property of Example 1.3 in transit. The control does not cover the same file at rest on the server, nor the question of who is allowed to open it at all. Those are Sessions 11 and 7.

6.4 Beyond the browser: VPNs and data on the move

TLS protects one connection between a browser and a server, but in the cloud remote access is the default: employees reach the service over the Internet, often from their own devices, at home or in public places. Comer (2021) gives three principles for this situation (Comer, 2021, pp. 240–241). The first is to keep all communication confidential. On a Wi-Fi network an outsider can capture every packet. The answer is a *virtual private network* (VPN), which forms an encrypted connection to the organisation's cloud data centre and routes all of a device's packets through it. Nothing then leaks around the tunnel, and the organisation can restrict Internet communication. A VPN thus protects the whole connection where TLS protects one application's. The second is to protect and isolate business data: a laptop or phone can be lost, so data stored on a device is encrypted at rest with whole-disk encryption, which is the at-rest control of Session 11 on the endpoint. The third is to enforce workflow security, so that the security policy for a piece of data travels with the data as it moves between cloud and device. Confidentiality must therefore follow the data across every hop: network, server and endpoint.

6.5 Limitations of encryption

Encryption helps only when it answers the threat that was actually identified, and only when the keys are kept apart from the data they protect, with separate access to each (CSA, 2024, p. 229). Against an attacker who obtains the data through the application, for example by SQL injection, it achieves nothing, because the application decrypts the data as part of answering, exactly as designed. The same limitation generalises: encrypting data in transit does not fix a weak password, encrypting data at rest does not fix an application that leaks it, and a key stored next to the data it protects is barely a key. Encryption is therefore one control among many, effective only together with the access control of Session 7 and the application security of Session 8, and the rule of Chapter 1 applies: name the threat first, then choose the control that meets it.

Reading for this chapter. CSA Security Guidance, Domain 9, §9.1.2 (Data States, pp. 210–211) and the encryption and key-management guidance of Domain 9 (§§9.2.3–9.2.5, pp. 216–221, and the recommendations, p. 229); Comer, Chapter 17, §17.9 (Protecting Remote Access: VPNs, whole-disk encryption and workflow security, pp. 240–241); Jøsang, Chapter 4 (Cryptography, pp. 63–98) and Chapter 5 (Key Management and PKI, pp. 99–116). The Microsoft Learn module is *Describe Azure security* (encryption in transit and Azure Key Vault); students enforce a minimum of TLS 1.2.

6.6 Summary

Data is at rest, in motion or in use (Definition 6.1), and each state needs its own control; this session covers data in motion, which the plain-HTTP Nextcloud left unprotected. Symmetric encryption is fast but needs a shared key, whereas asymmetric encryption solves key exchange and enables signatures but is slow (Definition 6.2). TLS combines them (Definition 6.4): an asymmetric handshake, authenticated by a certificate within a public-key infrastructure (Definition 6.3), agrees a symmetric session key that encrypts the traffic, and the result gives confidentiality, integrity and server authentication for data in transit (Example 6.1). A current version, keys in a managed store and a clear view of the scope are required, because TLS guards the channel, not the data at rest or in use behind it, and encryption aligned to the wrong threat protects nothing. Session 7 covers who is on the other end of the channel and what they may do.

Self-check

Sketch answers follow the exercises.

Exercise 6.1 (The three data states)**.** Name the three states of data and give, for each, one example from the Nextcloud service and the kind of control that fits it.

Exercise 6.2 (Why TLS uses both cipher types)**.** Why does TLS use both asymmetric and symmetric encryption rather than just one? What is each family good and bad at?

Exercise 6.3 (Trusting a server public key)**.** A browser receives a public key claiming to be the server's. What stops an attacker from supplying their own key instead, and what is that mechanism called?

Exercise 6.4 (http to https as threat–risk–control)**.** Formulate, in the vocabulary of Definition 1.3, the threat, risk and control for Nextcloud served over plain HTTP on a shared network, and name one thing the control does not protect.

Exercise 6.5 (Encrypted at rest, still stolen)**.** A database is encrypted at rest, yet an attacker steals customer records through an SQL-injection flaw in the web application. Why did the encryption not help, and what does this imply for choosing controls?

Sketch answers. (1) At rest: `salary_list.pdf` on the server's disk, protected by storage encryption and backups (Session 11); in motion: the same file downloaded over the network, protected by TLS (this session); in use: the file open in the application's memory, protected by access controls (Session 7). (2) Asymmetric encryption lets the parties agree a key over an untrusted network (the public key may travel in the open) but is slow; symmetric encryption is fast enough for bulk data but needs a shared key first. TLS uses asymmetric encryption once to establish a symmetric session key, then symmetric encryption for the data, each where it is strong. (3) The public key arrives inside a certificate signed by a certificate authority the browser already trusts, so the browser can verify that the key belongs to that domain; the system is the public-key infrastructure (PKI). Without it, an attacker in the middle could

supply their own key. (4) Threat: someone on the network path reads files and passwords sent in clear HTTP. Risk: on an untrusted network, likelihood real, impact high (confidentiality of data and credentials). Control: serve over HTTPS so that TLS encrypts the transport and the certificate authenticates the server. It does not protect the same data at rest on the server or decide who may access it. (5) The application decrypts the data to answer requests, so a flaw that makes it answer the attacker returns plaintext: at-rest encryption is bypassed, not broken. Controls must be chosen against the actual threat; encryption is one layer and does not substitute for secure application code or access control.

Chapter 7

Hardening and access control

The firewall of Session 4 decides where traffic may come from, but it says nothing about who is on the other end or what they may do once connected. This chapter covers authentication (proving who one is), authorisation (deciding what one may do) and least privilege. These questions weigh more in the cloud than elsewhere, because the whole infrastructure is managed through a web console and an API reachable from the Internet, so the account that controls it is the way in.

7.1 Identity as the new perimeter

The CSA Security Guidance calls identity and access management the new perimeter of the cloud, and the reason is this consolidation: a provider gathers all administration into one web console and one API, and both are reachable from the Internet (CSA, 2024, pp. 103–104). Often nothing more than a username and a password stands in front of them, and most breaches of cloud-native systems accordingly begin with a failure of identity management rather than with a network attack (CSA, 2024, p. 104). A stolen administrator credential is therefore no longer a foothold inside a building that must still be crossed; it can be control of everything at once. The perimeter formed by IAM covers authentication as well as authorisation, every kind of actor (people, systems and code) and every point an attacker can reach, from a phished password to a key left in a published container build file (CSA, 2024, p. 116). Five terms supply the vocabulary (CSA, 2024, pp. 105–106).

Definition 7.1 (Core IAM terms)**.** An **entity** describes a unique actor: a person, device, application or system. An **identity** describes the entity's verifiable expression within a namespace; one entity may hold several. **Authentication** describes the process of verifying an identity. **Authorisation** describes the decision to permit or deny an authenticated identity access to a resource. An **entitlement** describes the mapping of identities to authorisations, often with conditions (identity X may access resource Y when attributes Z hold); a table of these is an *entitlement matrix*.

Authentication and authorisation must be kept apart, because a system can get the first right and the second wrong: it identifies a user correctly and then grants far more access than the task needs. Least privilege exists to prevent that second failure, and the entitlement matrix, written down and reviewed, is the document that shows whether it holds.

7.2 Authentication and multi-factor authentication

Authentication rests on *factors*, that is, categories of evidence for a claim of identity. There are three, and combining them is the most effective identity control available.

Definition 7.2 (Multi-factor authentication)**. Multi-factor authentication** (MFA) describes the requirement of evidence from more than one independent category: *something one knows* (a password or PIN), *something one has* (a phone, hardware token or key) and *something one is* (a biometric). A stolen password alone is then insufficient, because it satisfies only one factor.

The cloud raises the need for MFA for two reasons (CSA, 2024, pp. 112–113). The first is *broad network access*, the second essential characteristic of Session 1: cloud services are reached over the Internet, so a lost or stolen credential can be used from anywhere, and the attacker no longer needs to be on the local network. The second is single sign-on across federated services, because one compromised set of credentials then opens many services at once. The Guidance accordingly ranks MFA among the most effective measures against account takeover, and a cloud account guarded by a password alone is a high risk (CSA, 2024, p. 113). The factors differ in strength: hardware tokens and phishing-resistant methods such as FIDO passkeys are stronger than a code read from an app, which in turn is stronger than a password alone (CSA, 2024, pp. 113–114). MFA is also the basis of *conditional access*, in which the system weighs the circumstances of a login, such as an unknown device, an unusual country, or the impossible travel of two logins from distant places minutes apart, and demands more assurance, or refuses, when they look wrong (CSA, 2024, pp. 106, 116).

Real-world case: Colonial Pipeline, 2021

The largest fuel pipeline in the United States was shut down for days, causing shortages along the East Coast, after a ransomware crew entered through a single account. The account belonged to a legacy virtual private network, its password had appeared in a batch of leaked credentials elsewhere, and it had no multi-factor authentication. One reused password without a second factor was sufficient. With MFA the stolen password alone would have failed (Definition 7.2). MFA on every account that reaches inside is a baseline, not advice.

7.3 Authorisation: RBAC, PBAC and ABAC

Once an identity is authenticated, authorisation decides what it may do, and three models express that decision, in rising order of flexibility (CSA, 2024, pp. 105, 118–119).

Definition 7.3 (Authorisation models)**. Role-based access control** (RBAC) describes granting permissions to *roles* and assigning identities to roles (administrator, member). **Policy-based access control** (PBAC) describes expressing permissions as explicit policies evaluated at access time. **Attribute-based access control** (ABAC) describes deciding from *attributes* of the identity, resource and context (department, resource sensitivity, time, location, MFA status), which enables fine-grained, context-aware decisions.

RBAC is simple and familiar, and for a small service it is often sufficient. Its weakness is coarseness, because everyone in a role receives the same access regardless of circumstance. ABAC and PBAC add that circumstance: access only from a managed device, only during working hours, only if MFA was used. The Guidance therefore names PBAC supporting ABAC as the preferred model and recommends ABAC and PBAC over RBAC where the platform supports them (CSA, 2024, pp. 115, 122). Table 7.1 compares the three.

Whichever model is used, one principle governs it.

Definition 7.4 (Least privilege)**.** The principle of **least privilege** describes the rule that every identity is granted the minimum access its task requires, and no more. Excess privilege is a hidden liability rather than a convenience: it is the additional damage available to any mistake and to any attacker who takes the identity over.

Model	Decides from	Strength	Weakness
RBAC	the identity's *role*	simple, familiar, easy to audit	coarse: all in a role get the same access
PBAC	explicit *policies* evaluated at access time	central and expressive	policies must be written and maintained
ABAC	*attributes* of identity, resource and context	fine-grained, context-aware	complex to design and reason about

Table 7.1. Three authorisation models. All three express who may do what; they differ in what the decision is based on and in the granularity bought at the cost of complexity. The Guidance prefers ABAC/PBAC where the platform supports them.

Least privilege has already appeared twice under other names: as `root` versus `sudo` in Session 2, where administrative power is not held while it is not in use, and as the firewall's default-deny in Session 4, where no network access is granted that has not been justified. Here it becomes the shape of every authorisation decision.

7.4 Protection domains and the access control matrix

RBAC, ABAC and least privilege are the modern form of a model from operating-systems theory (Tanenbaum & Bos, 2015, pp. 602–604). A system holds *objects* to be protected (files, devices, processes, memory), each with a name and a fixed set of operations. A *protection domain* is a set of *(object, rights)* pairs: for one actor, which operations are permitted on which objects. With every domain as a row and every object as a column, the result is the *access control matrix*, the most general statement of who may do what to what. Every authorisation scheme in this chapter compresses that matrix so that it can be managed at scale. RBAC groups rows that share a pattern into roles. ABAC computes a cell's contents from attributes at access time rather than storing it. An entitlement matrix (Definition 7.1) is the access control matrix written down for review. Least privilege, lastly, is Tanenbaum and Bos's own rule for filling the matrix, the *Principle of Least Authority* (POLA), also called need to know: each domain holds the minimum objects and rights needed for its work.

The Linux machine of Session 2 already works this way. In UNIX and Linux a process's protection domain is defined by its *user and group identifiers* (UID and GID): at login the shell receives them from the password database, every process started from it inherits them, and the pair determines which files and devices may be read, written or executed. Changing the pair changes the domain, and two mechanisms for doing so carry into the cloud. First, each process has a user half and a kernel half, and a *system call* switches between them. Because the kernel half can touch physical memory and every disk that the user half cannot, a system call is a domain switch to a more privileged domain, the same trap that made virtualisation work in Session 2. Secondly, running a program marked *set-UID* gives it a different effective identity for the duration. This is the mechanism behind `sudo`, and the reason `sudo` is a privilege escalation to be granted sparingly.

7.5 Single sign-on and privileged access

Zero Trust validates every request, and in a service built from many microservices a naive implementation would therefore prompt the user for a password at every hop. Comer (2021) gives the example of one task, finding a colleague's email address, that involves two services and asks the user to log in at each step (Comer, 2021, pp. 237–238). The situation is worse when every service keeps its own user list and passwords, because each service then grants privileges on its own, with no coordination. The answer is a central *identity management* (IdM) system

offering *single sign-on* (SSO) (Comer, 2021, p. 238). The user has one set of credentials, held in one place together with their access rights, every service consults that system, and the system can return a digital capability that other services accept, so the user enters credentials once per task. SSO is therefore what makes per-request checking workable, and it is the cloud form of the *federation* at the centre of the Guidance's identity domain (CSA, 2024, pp. 107–109). In Azure the central system is Microsoft Entra ID, which is why accounts, MFA and conditional access are managed there rather than in each application.

The same centralisation enables a control for the accounts that matter most: *privileged access management* (PAM), identity management for the administrators themselves (Comer, 2021, pp. 238–239). There is no single master password that opens every system; each member of staff holds administrative rights only on the systems that person is responsible for. The PAM system logs all accesses and failed login attempts, and some systems alert a manager when a privileged identity is used in a suspicious way, for example to reach a system for which it holds only secondary responsibility, or several such systems within a short time. The Guidance treats the same topic under privileged user management and recommends the strongest available authentication and monitoring for these accounts (CSA, 2024, pp. 117–118). PAM is thus least privilege applied to the administrators themselves.

7.6 Credentials and the management plane

Two practical hazards remain. The first is *static credentials*: long-lived secrets such as API keys embedded in code or configuration, which are a permanent gift to an attacker who reads the repository. The Guidance recommends eliminating them as far as possible (CSA, 2024, pp. 119–120), and where a human needs elevated access, granting it *just in time* (JIT): requested for a session, approved, and revoked when the session ends, so that no permanent key exists to steal (CSA, 2024, p. 105). The second hazard is the management plane itself. Because it controls everything, administrative access to it gets the strongest treatment available (CSA, 2024, pp. 116–118): MFA without exception, least privilege enforced, and administrative logins restricted by location where possible. Session 9 adds that every action on it is logged and watched.

Example 7.1 (Hardening the Nextcloud administrator)**.** Nextcloud has one all-powerful administrator account. *Threat*: the administrator's password is phished or reused from a breach elsewhere, and an attacker logs in with full control of every user's files. *Risk*: the likelihood is real, because password reuse and phishing are everywhere, and the impact is maximal, because the account can read, alter and delete everything, affecting all three CIA properties at once. *Controls*, layered: MFA on the administrator account, so that a stolen password alone fails (Definition 7.2); only the access they need for ordinary members, not administrative rights (least privilege); and no single shared administrator login, so that each action can be traced to a person, which is also what makes the logs of Session 10 meaningful. No single control removes the threat, but together they raise its cost.

Reading for this chapter. CSA Security Guidance, Domain 5, in particular §5.1 (How IAM is Different in the Cloud, and Fundamental Terms, pp. 104–106), §5.2 (Federation, pp. 107–111), §5.3 (Strong Authentication & Authorization, pp. 112–118), §5.4 (IAM Policy Types, pp. 118–119) and §5.5 (Least Privilege & Automation, pp. 119–121); the domain runs pp. 103–123. Comer, Chapter 17, §§17.5–17.7 (Zero Trust, Identity Management and single sign-on, and Privileged Access Management, pp. 237–239); Tanenbaum and Bos, *Modern Operating Systems*, §9.3 (Protection Domains and the access control matrix, and their UNIX/Linux realisation in UID/GID, pp. 602–604). Optional deepening: Jøsang, Chapter 8 (User Authentication, pp. 165–190) and Chapter 9 (IAM—Identity and Access Management, pp. 191–214). The Microsoft Learn

module is *Describe Azure identity* (Microsoft Entra ID, MFA and conditional access); students enable MFA on their own account.

7.7 Summary

When a cloud is managed through Internet-facing consoles and APIs, identity is the perimeter (Definition 7.1), and its two questions are kept apart: authentication proves who the requester is, authorisation decides what the requester may do. Because broad network access turns a stolen credential into an easy account takeover, multi-factor authentication, something one knows, has and is (Definition 7.2), together with conditional access, is the baseline. Authorisation is expressed through RBAC, PBAC or ABAC (Definition 7.3), the context-aware models being preferred where available, and all of them serve least privilege (Definition 7.4), which is grounded in the access control matrix of operating-systems theory. In practice this means replacing static credentials by just-in-time access and giving the management plane the strongest controls. On the red thread, the Nextcloud administrator gains MFA and members lose privileges they never needed (Example 7.1), so every action can be traced to a person, which the monitoring and logs of Sessions 9 and 10 depend on. Session 8 turns to the application itself.

Self-check

Sketch answers follow the exercises.

Exercise 7.1 (Authentication vs authorisation)**.** Distinguish authentication from authorisation, and give an example of a system that gets the first right and the second dangerously wrong.

Exercise 7.2 (The three authentication factors)**.** Name the three authentication factors, give an example of each, and explain why MFA defeats a stolen password.

Exercise 7.3 (Why the cloud needs MFA)**.** Why does the CSA Guidance hold that the cloud increases the need for strong MFA? Tie the answer to one of the NIST essential characteristics from Session 1.

Exercise 7.4 (RBAC vs ABAC)**.** Contrast RBAC and ABAC, and state which the Guidance prefers and why. How does each relate to least privilege?

Exercise 7.5 (Shared admin account as threat–risk–control)**.** Write, in the vocabulary of Definition 1.3, the threat, risk and controls for a single shared Nextcloud administrator account protected only by a password.

Sketch answers. (1) Authentication verifies identity; authorisation decides permitted actions. A system authenticates a user correctly but then grants them administrator rights they do not need: correct identity, excessive authorisation. (2) Something one knows (password/PIN), something one has (phone/hardware token), something one is (biometric); MFA requires more than one category, so a stolen password satisfies only one factor and fails alone. (3) Broad network access: cloud services are reached over the Internet, so a lost or stolen credential leads to account takeover without the attacker being anywhere near the victim, and passwords alone cannot carry that risk. (4) RBAC grants permissions to roles and assigns identities to roles, simple but coarse; ABAC decides from attributes and context, fine-grained and context-aware. The Guidance prefers ABAC (and PBAC) where supported; both serve least privilege by granting only what the task and context require. (5) Threat: the shared password is phished or reused, giving an attacker full control of all files. Risk: high likelihood, maximal impact (all three CIA properties). Controls: require MFA on the admin account, apply least privilege to members, and replace the shared login with per-person accounts so that each action can be traced to a person.

Chapter 8

DevSecOps, WAF and secure applications

The machine, the network, the transport and the identities are secured, and the application can still be the hole. Session 6 showed why: an SQL-injection flaw hands an attacker the data through the front door, and no encryption or firewall helps, because the application itself does the leaking. This chapter therefore covers the application layer from two sides: security built into the development process (DevSecOps), and a shield in front of the running application (the web application firewall).

8.1 Application security in the cloud

Application security protects applications from threats across their whole life, from design to production, and the cloud changes it in three ways (CSA, 2024, p. 230). First, applications are built as groups of microservices and external services communicating over APIs, so the attack surface is spread across many small components and their interfaces rather than one monolith. Secondly, they are developed with rapid DevOps practices, which is a risk, since fast change can outrun review, and an opportunity, since automation can enforce security on every change. Thirdly, they are assembled from libraries written by others, much of it open source, which brings in *supply-chain* risk, because a vulnerability in a dependency one did not write is still a vulnerability in one's application. The countermeasure to the third is the *software bill of materials* (SBOM), an inventory of every component and dependency, produced by software composition analysis in the image pipeline (CSA, 2024, pp. 181–182). When a library is found vulnerable, the SBOM answers the one question that matters during an incident: whether and where the library is in use.

Real-world case: Log4Shell, 2021

In December 2021 a critical flaw was disclosed in Log4j, a logging library so common that it sat, often invisibly, inside a large share of the world's Java applications. A single crafted string written to a log could make a vulnerable system fetch and run attacker code: trivial to exploit and broad in reach. The main difficulty was not the patch but finding out where Log4j was used at all: most organisations did not know which of their applications and third-party products embedded it. A software bill of materials answers that question in advance. Knowing one's dependencies before the need arises is a control in its own right.

8.2 The secure development lifecycle

A flaw caught in design costs a fraction of the same flaw caught in production, so security work is moved earlier (CSA, 2024, pp. 231–232).

Definition 8.1 (Secure development lifecycle)**.** The **secure software development lifecycle** (SSDLC) describes the integration of security into every stage of building software: secure *design and architecture*, secure *build, integration and testing*, secure *deployment*, and *runtime defence and monitoring* of the application in production. Moving security work earlier is called *shifting left*.

Two practices give the lifecycle substance. The first is *threat modelling* at design time, a structured process to identify, assess and remediate threats before the system is built (CSA, 2024, p. 232). The Guidance recommends *STRIDE*, which sorts threats into six categories, Spoofing, Tampering, Repudiation, Information disclosure, Denial of service and Elevation of privilege (CSA, 2024, pp. 232–233), and thereby turns the open-ended threat question of Session 1 into a checklist that is hard to leave gaps in. The second practice is automated security testing in two forms. *Static* application security testing (SAST) examines the source code before deployment for security flaws and logic errors and scans for hard-coded credentials, keys and tokens before they enter the repository (CSA, 2024, p. 234); it produces false positives and therefore needs tuning. *Dynamic* testing (DAST) probes the running application from outside afterwards, emulating real attacks (CSA, 2024, p. 235). Static testing sees the code but not its behaviour, and dynamic testing sees the behaviour but not the code, so the two are used together, as symmetric and asymmetric encryption were in Session 6.

8.3 DevSecOps

The lifecycle becomes practical when it is automated into the pipeline that already builds and ships the software.

Definition 8.2 (CI/CD and DevSecOps)**.** A **continuous integration / continuous delivery** (CI/CD) pipeline describes the set of automated steps that build, test and deploy software as developers merge changes. **DevSecOps** (development, security and operations) describes embedding security into that pipeline, so that automated security checks run on every change rather than as a separate, occasional gate.

Comer (2021) describes the CI/CD machinery and two deployment strategies that make frequent releases safe (Comer, 2021, pp. 211–213). *Canary deployment* sends the new version to a small fraction of users first and observes before rolling on, and *blue/green deployment* keeps two environments so that a release can be switched over, and back, instantly. DevSecOps adds security to each phase: SAST runs during continuous integration, so that vulnerable code does not merge, and DAST runs during continuous delivery against the staged application (CSA, 2024, pp. 243–245). This closes a loop with Session 5, because the immutable infrastructure of the cloud-native world is what makes the pipeline trustworthy: every environment is built fresh from the same tested definition, so there is no drift between what was tested and what runs. The CSA framing, six pillars, is cultural as much as technical: security is a shared responsibility across development, operations and security, automated to minimise human error and measured for continuous improvement (CSA, 2024, p. 244). In contrast to a security team that reviews at the end, DevSecOps places the check where the change is made.

What lets a pipeline build the same tested definition every time is that the infrastructure itself is written down as text. *Infrastructure as code* (IaC) replaces hand-configuration, which no two engineers perform identically, with a machine-readable specification that provisions the environment automatically (Comer, 2021, pp. 121–122). Comer connects it to *zero touch provisioning*, bringing systems up with no manual steps, and separates an *imperative* specification,

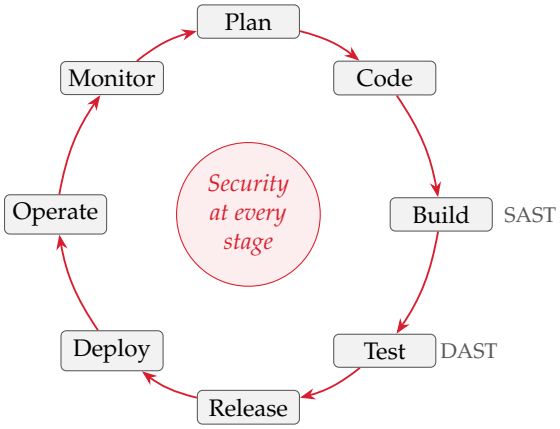

Figure 8.1. The DevSecOps loop. Development (plan, code, build, test) flows continuously into operations (release, deploy, operate, monitor) and back; DevSecOps embeds automated security at every stage rather than as a gate at the end, with SAST as the code is built and DAST against the deployed application.

which lists the steps to perform, from a *declarative* one, which states the desired end state and lets the tooling reach it, the same work-towards-a-goal style as the Kubernetes control plane of Session 5. Three security benefits follow. Consistency removes the configuration drift that breeds vulnerabilities. The specification is reviewable and version-controlled, so a change is visible, auditable and revertible, which Session 12 uses for compliance. And because the environment is rebuilt from the text rather than patched by hand, it is immutable in the sense of Session 5. IaC and pipelines are, however, themselves part of the attack surface and need the same access control as the application (CSA, 2024, p. 238).

8.4 The web application firewall

Secure development reduces the flaws that reach production, but it does not remove all of them, and it does nothing for flaws in third-party code one does not control. The running application therefore needs a guard of its own.

Definition 8.3 (Web application firewall)**.** A **web application firewall** (WAF) describes a component placed in front of a web application that inspects incoming HTTP requests at the application layer and blocks those that match patterns of attack, such as SQL injection, cross-site scripting and the application-layer floods of Session 3, before they reach the application.

Here the distinction of Session 4 applies: an NGFW secures the network broadly, a WAF secures one web application specifically, and they operate at different layers. The WAF is the control that answers the SQL-injection example of Session 6, because it inspects a request for the shape of an injection attempt and blocks it before the application can execute it. In the SSDLC it belongs to the runtime-defence phase, the external shield around a production application that complements secure code within. The Guidance accordingly treats it as a control that prevents attacks from arriving, not as a way to repair a flawed application or to excuse careless development (CSA, 2024, p. 247). On the students' service it runs in front of Nginx; in Azure it is the Application Gateway's WAF at Layer 7.

Example 8.1 (A WAF for Nextcloud)**.** The SQL-injection scenario of Session 6 now meets a control that fits it. *Threat*: a crafted request exploits an injection flaw, possibly in a third-party Nextcloud plugin, to read files from the database, bypassing TLS and the firewall because it arrives as an ordinary-looking HTTPS request. *Risk*: the likelihood is non-trivial, because plugins and dependencies carry such flaws, and the impact is high, because the confidentiality of every stored file is at stake. *Controls*, in depth: secure coding and dependency scanning in

the pipeline reduce the chance that the flaw exists (shift left); an SBOM shows at once whether a newly disclosed library vulnerability affects the service; and a WAF in front of Nextcloud inspects and blocks the malicious request at runtime even if the flaw is present. The layers are independent by design: the WAF catches what the pipeline missed, and the pipeline shrinks what the WAF must catch.

Reading for this chapter. CSA Security Guidance, Domain 10, in particular §10.1 (Secure Development Lifecycle, threat modelling and STRIDE, pre- and post-deployment testing, pp. 231–236), §10.4 (DevSecOps: CI/CD & Application Testing, pp. 243–248) and §10.5 (Serverless & Containerized Application Considerations, pp. 248–250); the domain runs pp. 230–252. Domain 8, §§8.1.4–8.1.5 (Software Composition Analysis and the Software Bill of Materials, pp. 181–182). Comer, Chapter 15 (DevOps: CI/CD, canary and blue/green deployment, pp. 207–214) and §§9.10–9.11 (infrastructure as code; declarative, imperative and intent-based specifications, pp. 121–122). Optional deepening: Jøsang, Chapter 11 (Secure by Design, pp. 243–270). The Microsoft Learn module for Session 8 (Azure Application Gateway and its WAF) is linked on Canvas.

8.5 Summary

The application is a layer no network control reaches, and the SQL injection of Session 6 shows why it needs its own defences. In the cloud those defences must cope with microservices and APIs, rapid change and third-party dependencies, the last tracked with a software bill of materials. Security is built in through the secure development lifecycle (Definition 8.1), which models threats with STRIDE and tests with SAST before deployment and DAST after, and it is made routine by embedding those checks in the CI/CD pipeline as DevSecOps (Definition 8.2), which the immutable, code-defined infrastructure of Session 5 keeps trustworthy. Because flaws still reach production and third-party code cannot be fixed at will, a web application firewall (Definition 8.3) shields the running application and blocks the injection attack at runtime (Example 8.1). Every control so far can still fail, and Session 9 therefore turns to detection.

Self-check

Sketch answers follow the exercises.

Exercise 8.1 (Shifting left)**.** What does shifting left mean, and why is a vulnerability cheaper to fix in design than in production?

Exercise 8.2 (SAST vs DAST)**.** Distinguish SAST from DAST. Why are they used together, and which earlier pair of complementary techniques does this resemble?

Exercise 8.3 (What DevSecOps adds)**.** Explain what DevSecOps adds to a CI/CD pipeline, and how the immutable infrastructure of Session 5 makes the pipeline's testing trustworthy.

Exercise 8.4 (NGFW vs WAF, revisited)**.** An NGFW and a WAF both inspect traffic. State the difference in one sentence, and name the one that blocks an SQL-injection attempt against a Nextcloud login.

Exercise 8.5 (A disclosed library flaw)**.** A newly disclosed vulnerability affects a popular open-source library. Which document shows immediately whether the application is affected, and which runtime control could block exploitation even if it is?

Sketch answers. (1) Shifting left moves security work earlier in the lifecycle (design and build rather than production); a flaw caught in design is fixed by changing a plan, while the same flaw in production may mean an incident, an emergency patch and lost data. (2) SAST examines source code before deployment to find flaws; DAST probes the running application from outside afterwards. Each has a blind spot, SAST seeing code but not behaviour and DAST behaviour but not code, so they are combined, as symmetric and asymmetric encryption are in Session 6. (3) DevSecOps embeds automated security checks (SAST in CI, DAST in CD) into every change rather than an occasional gate; because immutable infrastructure builds every environment fresh from one tested definition, there is no drift between what was tested and what runs. (4) An NGFW secures the network broadly with application awareness; a WAF secures one web application by inspecting HTTP requests; the WAF blocks the SQL-injection attempt. (5) A software bill of materials (SBOM) shows whether and where the library is used; a web application firewall could block exploitation of the flaw at runtime even if the vulnerable library is present.

Chapter 9

Security monitoring and IDS

Every control of Sessions 4 to 8 can fail: a firewall can be misconfigured, an identity phished past its MFA, a WAF bypassed, a dependency flawed. Prevention is therefore necessary but never sufficient, and monitoring assumes that a breach will happen and arranges to discover it quickly. Its foundation is the baseline of Session 2, because before something abnormal can be recognised, normal must be known. This chapter covers what is collected and how a large volume of it becomes a timely alert.

9.1 Monitoring in the cloud

The CSA Security Guidance names four sources of complexity in cloud monitoring (CSA, 2024, pp. 124–125). The first is the *management plane*, which controls all administrative actions and must be watched closely because it grants access to everything. The second is *velocity*: changes occur at high speed, so automated responses are needed to keep up. The third is *distribution and segregation*, which limits the blast radius of a breach but requires some centralisation of logs to see the whole estate. The fourth is *cloud sprawl* across workload types and providers. Speed matters most: attacks on cloud systems are often scripted, so data can be stolen or exposed within seconds or minutes of the first break-in (CSA, 2024, p. 262), and detection of the most serious activities therefore works in real time where possible, and in any case within minutes. The shared-responsibility model applies here as elsewhere: the provider monitors some layers and the customer others, and knowing which is the starting point (CSA, 2024, p. 124).

9.2 Events, alerts and telemetry

Definition 9.1 (Event and alert)**.** An **event** describes anything that happens in a system or network and can be recorded (a login, an API call, a file access); most events are normal. An **alert** describes a message that demands attention, produced by analysing events, raised when events match a rule or pattern that signals a potential threat. Alerts are not events: the task of monitoring is to turn a great many events into a few meaningful alerts.

Real-world case: Target, 2013

Around 40 million payment cards were stolen from the US retailer Target over the holiday season. The attackers first entered through credentials taken from a third-party heating and refrigeration contractor with network access, then moved to the point-of-sale systems. Target's malware-detection system generated alerts about the intrusion, but nobody acted on them in time. Detection is half the task: an alert nobody responds to is no better than no alert. The response process of Session 12 exists for this reason.

Alerts are fed by *telemetry*, the sources that show what is happening in the environment (CSA, 2024, p. 127), and two sources matter most (CSA, 2024, p. 128). *Management plane logs* record control actions, that is, who accessed the infrastructure, what they did and when, so an attacker on the console leaves the first trace here. *Service and application logs* record the activity of individual services: authentication attempts, API access, network traffic, service-specific events. Nextcloud already produces both kinds, namely the access log of Session 3 and the system journal of Session 2, and monitoring means watching them continuously instead of reading them afterwards.

9.3 Detection: from telemetry to alert

Watching everything by hand does not scale, so detection narrows telemetry to what deserves attention.

Definition 9.2 (Intrusion detection and prevention)**.** An **intrusion detection system** (IDS) describes a system that monitors telemetry for signs of attack and raises an alert. An **intrusion prevention system** (IPS) describes a system that does the same but sits *inline* in the traffic path, so that on a match it can block the traffic rather than merely report it. Either may work by *signature* (matching known attack patterns) or by *anomaly* (flagging departures from an established baseline of normal behaviour). In the cloud these functions are increasingly delivered as services, for example SIEM for correlating logs and CSPM for detecting insecure configurations, rather than by a single appliance.

The distinction is the detect-versus-prevent choice of Session 4. An IDS is the detection mode of the network: passive and out-of-band, it observes a copy of the traffic and cannot stop an attack, so it cannot break legitimate traffic, but it is only as fast as the humans who act on its alerts. An IPS is the prevention mode: inline, it can drop a malicious packet in real time, but a false positive then blocks a real user, and the device sits in the critical path. Prevention stops more but risks more, so the two are used together: an IPS blocks the clearly malicious, and an IDS surfaces the merely suspicious for a human to judge.

Figure 9.1. IDS versus IPS. An IDS observes a copy of the traffic and alerts; an IPS sits in the path and can block. Detection is safe but passive; prevention acts but can break legitimate traffic.

The pattern the Guidance recommends is *cascade-and-filter*: a sequence of detection mechanisms applied to the incoming streams, each narrowing the data further (CSA, 2024, pp. 139–140). A first filter flags whatever looks broadly suspicious, for example odd login attempts or unexpected network traffic; later filters narrow toward known attack patterns; and a final filter triggers an alert or a response action if a threat is confirmed. The same cascade manages cost, because most raw logs stay cheaply where they are and only pertinent alerts and selected logs go to the security team (Session 10). To decide what to look for, the Guidance points to the MITRE ATT&CK framework, a catalogue of the tactics and techniques attackers use, which connects a given indicator with the stage of an attack it belongs to (CSA, 2024, p. 127). It names key indicators to watch closely, namely unusual IAM or network security activity, especially in unused regions (CSA, 2024, p. 127), and its list of detectors adds unauthorised API calls, management console logins without MFA, any IAM policy change and any use of the root

account (CSA, 2024, p. 142). These are precisely the management-plane and identity activities of Sessions 4 and 7. A further technique cuts detection time sharply: a *canary* or *honey token* is a decoy credential or resource that no legitimate process should touch, so any interaction with it indicates an attempted or actual breach, with almost no false positives (CSA, 2024, p. 142).

Comer (2021) adds two points on the mechanics. A monitor at its simplest is a process that loops forever, checks each instance, raises an alert if one fails to respond, and waits thirty seconds before repeating (Comer, 2021, p. 182). That loop does not fit cloud-native systems, however (Comer, 2021, p. 183). A traditional distributed application runs a known number of instances in known places, so an instance that stops responding means trouble. A cloud-native application, in contrast, starts and stops instances all the time, and the orchestrator decides where each one runs, so a monitoring tool must not raise an alarm every time an instance disappears because the scale was reduced. This is the moving-target problem of Session 5 seen from the monitoring side, and it is why cloud monitoring relies on management-plane and orchestration signals rather than on a static list of hosts.

The anomaly half of detection is *security analytics* (Comer, 2021, pp. 239–240). Analytics software learns what normal behaviour looks like and then watches for exceptions, for example an employee touching data or services they never touch, or acting at an unusual hour; this is the Session 2 baseline turned into a detector. Good analytics is context-aware, because an access can be individually authorised and still suspicious when it does not fit its usual pattern: the right person, the right permission, the wrong context.

9.4 Fast path and slow path

Whether monitoring is timely depends on one further design choice, and it follows from the speed problem above. The Guidance separates two processing tracks (CSA, 2024, p. 135). The *slow path* carries logs, which can arrive up to fifteen minutes after the fact, and that delay is acceptable for investigation and forensics. The *fast path* carries events, which are analysed and turned into alerts almost at once, and it is necessary for high-risk activity, especially on the management plane, where fifteen minutes is too long. Fast-path events therefore detect and alert on high-risk activity as it happens, while slow-path logs serve the deeper analysis that follows. A system that puts everything on the slow path discovers its intrusions after they have finished.

Example 9.1 (Watching the red thread)**.** *Threat*: an attacker who has passed the earlier controls, for example with a phished credential, begins to act: logging in from an unrecognised country, making unusual administrative changes, touching files no ordinary user would. *Risk*: without monitoring the likelihood of noticing is low and the impact grows with every undetected hour. *Controls*: alerts on authentication failures and impossible-travel logins (fast path); the management plane and IAM changes watched as key indicators; a canary file whose every access is a definite alarm; and the access and system logs kept flowing for the timeline Session 12 requires. None of this prevents the intrusion; it turns a silent compromise into a detected incident.

Reading for this chapter. CSA Security Guidance, Domain 6, in particular §6.1 (Cloud Monitoring, pp. 124–127, including Logs & Events, Alerts & Monitoring, Timeliness and Key Indicators), §6.2 (Cloud Telemetry Sources, pp. 127–133), §6.3.3 (Monitoring Strategy, including the fast and slow paths, pp. 135–137) and §6.4 (Detection & Security Analytics, including the cascade-and-filter pattern, detectors, and canaries, pp. 137–142); the domain runs pp. 124–145. Comer, Chapter 13, §§13.3–13.4 (periodic monitoring and the added complexity of monitoring cloud-native applications, pp. 182–183) and Chapter 17, §17.8 (security analytics and context-aware anomaly detection, pp. 239–240); and the MITRE ATT&CK® Cloud Matrix. Optional deepening:

Jøsang, Chapter 16 (Cyber Operations, pp. 337–354). The Microsoft Learn module is *Describe Azure monitoring* (Microsoft Defender for Cloud and Sentinel).

9.5 Summary

Because every preventive control can fail, monitoring assumes a breach and detects it quickly, on the basis of the Session 2 baseline. Cloud monitoring is shaped by two facts: attacks are fast and automated, and the management plane is the highest-value target. Telemetry, that is, management plane logs and service logs, is turned from many events into few alerts (Definition 9.1). Detection (Definition 9.2) does this through signature and anomaly methods delivered by SIEM and CSPM, arranged as a cascade-and-filter, guided by MITRE ATT&CK toward key indicators (unauthorised API calls, IAM policy changes, network configuration changes) and sharpened by canaries. Timeliness depends on routing high-risk activity to the fast path and forensic logs to the slow path. Nextcloud is now watched (Example 9.1), and Session 10 covers whether the record of what it saw can be trusted.

Self-check

Sketch answers follow the exercises.

Exercise 9.1 (Event vs alert)**.** Explain the difference between an event and an alert, and why "turning events into alerts" is the central problem of monitoring.

Exercise 9.2 (The two key telemetry sources)**.** Name the two most important cloud telemetry sources and state what each records. Why does the Guidance insist that monitoring cover the management plane?

Exercise 9.3 (Signature vs anomaly detection)**.** Distinguish signature-based from anomaly-based detection. Which one relies on the Session 2 baseline, and why?

Exercise 9.4 (Fast path vs slow path)**.** What are the fast path and the slow path, and which should carry a suspicious administrative action on the management plane, and why?

Exercise 9.5 (The canary token)**.** Explain how a canary (honey token) produces an alert with essentially no false positives, and give one that could be placed in a Nextcloud deployment.

Sketch answers. (1) An event is anything that happens and can be recorded, mostly normal; an alert is a message that demands attention, raised when events match a rule. Because there are vastly more events than incidents, the value of monitoring lies in filtering the many events down to the few meaningful alerts. (2) Management plane logs (who accessed the infrastructure, what they did, when) and service/application logs (authentication attempts, API access, network traffic, service events). The management plane controls everything, so losing it is the worst case; it must be watched so that an attacker there is seen immediately. (3) Signature-based detection matches known attack patterns; anomaly-based detection flags departures from normal. Anomaly detection relies on the baseline, because abnormal is only defined against a recorded picture of normal. (4) The fast path analyses events in near real time for high-risk activity; the slow path handles logs with delay for forensics. A suspicious management-plane action belongs on the fast path, because a delay of minutes is too long for the highest-value target. (5) A canary is a decoy no legitimate process should touch, so any access is almost certainly malicious; for example a fake `admin_passwords.txt` or an unused decoy user account in Nextcloud, whose every access raises a definite alarm.

Chapter 10

Logging and log integrity

Session 9 observed the signals; this chapter asks whether the record of them can be believed. Logs have two roles. Operationally they show how a system behaves; in security they are evidence, namely the timeline of an attack, the record of who did what, the material an incident responder, an auditor or a court relies on. That second role raises a question monitoring alone does not answer: what happens to the evidence when the machine producing it is compromised, since an attacker who can edit or delete the logs on a host erases their own trace. This chapter covers collecting logs so that they are useful and protecting them so that they can be trusted, which is the integrity property applied to the record itself.

10.1 Log sources

Session 9 named the telemetry that matters, and the same sources are now captured deliberately (CSA, 2024, p. 128). *Management plane logs* record control-plane actions, that is, who accessed the infrastructure, what they changed and when, so an attacker on the console is seen here first. *Service and application logs* record the activity of individual services: authentication attempts, API calls, network traffic, errors. On the red thread these are Nextcloud's own access and audit logs, the Nginx access log of Session 3, the system journal (`journalctl`) of Session 2 and the firewall's record of what it dropped. Because an event that was never logged cannot be investigated, the first rule of logging is to know what each source captures and to make sure that the security-relevant sources are actually recorded.

10.2 Logs, events and the content of a log line

The Guidance separates logs from events, and the separation decides how each is used (CSA, 2024, p. 125). A *log* is a relatively complete record of activity (create, read, update, delete), detailed and stored persistently, but often delivered in batches and therefore delayed. An *event* typically records only a change (create, update, delete), is not retained unless explicitly saved, and becomes available within seconds, which is why it can trigger a real-time alert from a service such as AWS GuardDuty, Microsoft Sentinel or GCP Security Command Center. Events thus give timely notice that something happened, and logs give the depth to investigate what it was.

Three lines from the running example, slightly shortened, show what a log line contains:

```
# Nginx access log -- a normal request, then a probe
192.0.2.44 - - [06/Jul/2026:14:22:07] "GET /apps/files" 200 5120
203.0.113.9 - - [06/Jul/2026:14:22:41] "GET /login?user=admin'-" 403
# System journal -- a failed SSH login
Jul 06 14:23:19 nc-vm sshd[2011]:  Failed password for invalid
  user admin from 203.0.113.9 port 51234 ssh2
# Azure Activity Log -- a management-plane change
operation:  Microsoft.Network/networkSecurityGroups/write
caller:  attacker@example.com   result:  Succeeded
```

Each line carries the same essentials: who (source address or caller), what (the request or operation), when (the timestamp) and the outcome (the `200`, `403` or `Succeeded`). The first block shows an ordinary file fetch followed by an SQL-injection probe against the login, refused with a `403`. The second shows a failed SSH login; one is noise, but fifty a minute is a brute-force attempt. The third is a change to a network security group on the management plane, one of the key indicators of Session 9. Each line on its own is mundane; gathered and correlated, such lines become the timeline of an incident.

In the cloud these sources have concrete names. On Azure the management plane log is the *Activity Log*, which contains subscription-level events such as a resource being modified or a VM started; it is enabled by default, retained for ninety days, and is where forensics after a breach begins. Per-resource *Resource Logs* (formerly called diagnostic logs) record the internal activity of a service but are not collected by default, because a diagnostic setting must first be enabled or an agent installed, and this gap commonly leaves exactly the logs one wants missing when they are needed. Both kinds can be routed to *Log Analytics*, an aggregation-and-query service (queried in KQL) that plays the role of the SIEM below, or exported to a third-party SIEM through an Event Hub.

10.3 Central collection

Logs scattered across many hosts are hard to search and easy to lose, and in the cloud, where instances are ephemeral (Session 5), a log left on a host vanishes with the host. For exactly this reason the Guidance advises that a workload's logs leave it for a central store as soon as they are written (CSA, 2024, p. 181).

Definition 10.1 (Log aggregation and SIEM)**. Log aggregation** describes the practice of forwarding logs from many sources to a central repository for storage, search and analysis. A **security information and event management** (SIEM) system describes such a repository specialised for security: it collects logs across environments, correlates them to identify potential incidents, and supports the detection and investigation of Session 9.

The Guidance calls the resulting architecture a *cascading log architecture*, in which logs flow from one layer to the next (CSA, 2024, p. 134). Each environment (development, test, production) sends its logs to one central store. That store collects everything and passes the subset that matters for security on to a separate, well-protected audit environment, where it is kept for the long term, and from there the logs feed a SIEM that analyses across all environments.

Two design points from Session 9 carry over. Cascade-and-filter applies here as cost control (CSA, 2024, pp. 133–135): pertinent alerts and selected logs go to the security team, bulk raw logs stay cheaply in place, and rarely-used logs move to lower-cost storage for retention rather than being discarded. The fast-path/slow-path split applies too, since some logs feed near-real-time alerting and most feed slower analysis. *Retention*, that is, how long logs are kept, balances operational needs, regulatory requirements and cost (CSA, 2024, p. 133): an intrusion may be discovered months after it began and compliance obligations (Session 12) set minimum periods, whereas prolonged retention raises storage cost and has privacy implications. Some log data

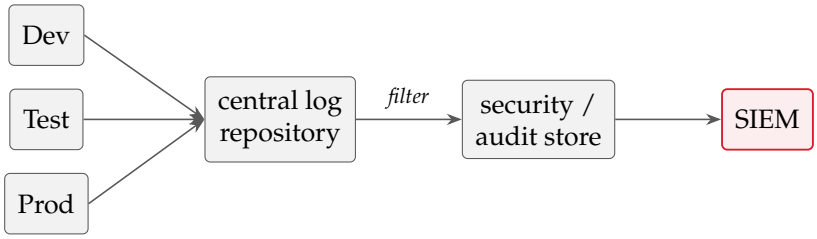

Figure 10.1. Cascading log architecture. Each environment forwards to a central repository; the security-relevant subset is filtered into a retained audit store and fed to the SIEM, the same cascade-and-filter as in Session 9, now applied to storage and cost.

should also leave the cloud altogether, both so that it survives a failure of the platform and because regulators may require it (CSA, 2024, p. 133), and, as the next section shows, this is an integrity control as well as a resilience measure.

10.4 Log integrity

Everything above assumes that the logs are an honest record, and an attacker's assumption is the opposite.

Definition 10.2 (Log integrity)**. Log integrity** describes the assurance that log records have not been altered or deleted since they were written. It is the integrity property of the CIA triad applied to the audit record itself: without it, logs can be edited to hide an attacker's actions and cannot be trusted as evidence.

On many intrusions, one of the attacker's first moves after gaining control of a host is to edit or wipe its logs, removing the record of how they entered and what they did. If the logs exist only on the compromised machine, the attacker rewrites the record freely, so the record must be placed where the compromised host cannot reach it. Three measures follow. Logs are shipped off-host to the central store of Definition 10.1 as they are produced, so that a copy exists beyond the attacker's control the moment the event happens. They are stored append-only or write-once, so that even with access they can be added to but not edited or deleted. And access to the log store is restricted under the least-privilege rules of Session 7. The principle is the one that makes off-site backups matter in Session 11: a safeguard under the attacker's control is no safeguard. Centralised collection is therefore itself an integrity control, because logs that left the host at once are logs the host's attacker never reached.

10.5 Logs as evidence

The reason integrity is worth this effort is what logs become during an incident. The Detection and Analysis phase of Session 12 builds a timeline of the attack from the logs (CSA, 2024, p. 255), and a timeline assembled from records that might have been tampered with proves nothing. Incident response therefore speaks of *chain of custody*: evidence is documented and preserved so that its integrity can be demonstrated later, to management, to regulators or in legal proceedings (CSA, 2024, pp. 255, 266). A snapshot (Session 2) and an off-host, append-only log have the same forensic value, a trustworthy record of a moment the attacker could not revise. On a real system the logs are therefore worth exactly as much as the confidence that they still state what happened.

Example 10.1 (Protecting the record)**.** An attacker gains a foothold on the Nextcloud VM. *Threat*: to cover their tracks, they delete the Nginx access log and edit the system journal, erasing the evidence of the intrusion. *Risk*: if the only copy is on the host, the likelihood of successful

tampering is high and the impact severe, because the incident may never be reconstructed and cannot be proven if it is. *Control*: forward all security-relevant logs off-host to a central store as they are written, held append-only, with tightly restricted access. The attacker now controls the host but not the record: the copy that left in real time survives, the timeline can be rebuilt, and the chain of custody holds. The control protects the integrity of the evidence rather than its secrecy, and it is what makes the response of Session 12 possible.

Reading for this chapter. CSA Security Guidance, Domain 6, §6.1.1 (Logs & Events, pp. 125–126), §6.2 (Cloud Telemetry Sources, pp. 127–133), §6.3.1 (Log Storage & Retention, p. 133) and §6.3.2 (Cascading Log Architecture, pp. 134–135); Domain 11, §11.1.1 (the incident-response checklist: building the attack timeline and chain of custody, pp. 254–256) and §11.3.4 (Cloud System Forensics, pp. 265–267). Optional deepening: Jøsang, Chapter 16 (Cyber Operations, pp. 337–354), on logging for detection and forensics. The Microsoft Learn module is *Describe Azure monitoring* (Azure Monitor, Activity Logs, Log Analytics and diagnostic settings).

10.6 Summary

Logs are both an operational tool and security evidence, which raises the question whether the record can be trusted when the host producing it is compromised. The sources are those of Session 9, management plane logs and service and application logs, and concretely the records of Nextcloud, Nginx, the journal and the firewall. Because scattered logs are easily lost and ephemeral hosts take their logs with them, logs are aggregated centrally (Definition 10.1) into a SIEM, with cascade-and-filter for cost, fast and slow paths for timeliness, and retention set by investigation needs and compliance. Log integrity (Definition 10.2) requires that the record leave the host as it is written and be held append-only with restricted access, so centralised collection is itself an integrity control. Under chain of custody the logs then become the attack timeline and the evidence for the response of Session 12 (Example 10.1). Session 11 turns from proving harm to surviving it: resilience, RAID and backup.

Self-check

Sketch answers follow the exercises.

Exercise 10.1 (The two roles of logs)**.** Give the two roles logs play, operational and evidential, and one Nextcloud log source relevant to each.

Exercise 10.2 (Central collection for ephemeral hosts)**.** Why is centralised log collection especially important for ephemeral cloud workloads (Session 5)?

Exercise 10.3 (Log integrity and the CIA triad)**.** Define log integrity in terms of the CIA triad, and explain why logs kept only on a host fail it once the host is compromised.

Exercise 10.4 (Protecting log integrity)**.** Name three concrete measures that protect log integrity, and state why centralised collection is an integrity control and not only a convenience.

Exercise 10.5 (Chain of custody)**.** What is chain of custody, and why does it make off-host, append-only logging matter for the incident response of Session 12?

Sketch answers. (1) Operational: understanding the system and reconstructing behaviour, for example the Nginx access log or the system journal. Evidential: the timeline and proof of who did what during an incident, for example Nextcloud's audit log and the management plane logs. (2) Ephemeral instances disappear with demand, taking locally stored logs with them; forwarding logs to a central store as they

are written preserves them beyond the life of the host. (3) Log integrity is the integrity property of CIA applied to the audit record, the assurance that it has not been altered or deleted. Logs kept only on the host fail it because an attacker who controls the host can edit or wipe them, destroying the record of their own actions. (4) Ship logs off-host as they are produced; store them append-only or write-once; restrict who may access the log store (least privilege). Central collection is an integrity control because logs that left the host immediately are beyond the reach of the host's attacker. (5) Chain of custody is the documented preservation of evidence so that its integrity can be demonstrated later (to management, regulators or a court); off-host, append-only logs provide a trustworthy, untampered record, which is what lets the attack timeline in Session 12's Detection and Analysis phase stand as proof.

Chapter 11

Resilience, RAID and backup

Most controls in this course reduce the *likelihood* of a bad event: the firewall makes intrusion less likely, MFA makes account takeover less likely, the WAF makes exploitation less likely. Resilience is different. It accepts that a disk dies, a data centre floods, ransomware encrypts the files or an administrator deletes the wrong directory, and it reduces the *impact*, so that the service keeps running or recovers quickly. Two tools carry it: redundancy, which lets a system survive a component failure, and backup, which returns the system to a good state. Because the two are constantly confused, keeping them apart is half the chapter.

11.1 Availability revisited

Session 3 showed that chaining components multiplies their availabilities down, so redundancy must be added on purpose, and Session 2 introduced the geography, availability zones and region pairs, that makes large-scale redundancy possible. The CSA Security Guidance describes resilience built on that geography as layered (CSA, 2024, pp. 270–271). Single-region resilience, with autoscaling, load balancing and backup, is where most applications begin. Multi-region resilience, with parallel deployments across regions, adds fault tolerance against regional outages at the cost of duplicated resources and data synchronisation. The design goal at every level is a small *blast radius*, so that one failure takes down as little as possible. A deployment spread across availability zones survives the loss of one. A service whose data is backed up survives the loss of its storage. A system built from immutable images (Session 5) survives the loss of an instance by replacing it.

11.2 Redundancy and RAID

The oldest form of redundancy works at the level of storage, and it is the subject of this week's laboratory. In a data centre, storage is pulled out of individual servers into centralised facilities (Session 2), and RAID is part of how that storage survives the failure of the physical disks beneath it.

Definition 11.1 (RAID)**. RAID** (redundant array of independent disks) describes the combination of several physical disks into one logical volume for performance, redundancy, or both. **RAID 0** *stripes* data across disks for speed but adds no redundancy, so one disk failing loses everything. **RAID 1** *mirrors* data across disks, so a copy survives a disk failure. **RAID 5** and **RAID 6** distribute data with *parity*, tolerating one or two failed disks respectively while using space more efficiently than mirroring.

RAID 0 is the trap in this list. Its name and its place among the redundant arrays suggest safety, but it offers striping for speed with no redundancy at all, so the array is more likely to

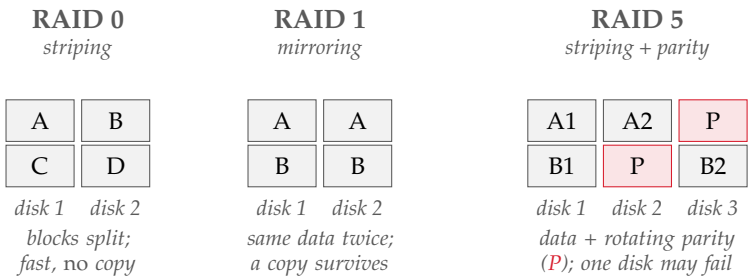

Figure 11.1. Three RAID levels. Striping (RAID 0) spreads blocks for speed with no redundancy; mirroring (RAID 1) keeps a full second copy; parity (RAID 5) distributes data and parity so that any one disk can be rebuilt. None of them is a backup.

lose data than a single disk, because any one of several disks failing destroys the whole. It suits scratch data one can afford to lose and nothing of value. A second rule applies to all RAID levels and is the most frequently confused idea in this area: *RAID is not a backup*. RAID protects against a disk failing, but it does nothing against a file being deleted, corrupted or encrypted by ransomware, because a mirror or parity array instantly and exactly copies that deletion to every disk. Redundancy therefore keeps a service running through hardware failure, and it cannot undo a mistake or an attack; that requires a return to an earlier state, which RAID cannot provide.

11.3 Backup

Definition 11.2 (Backup)**.** A **backup** describes a separate, point-in-time copy of data, kept so that the data can be restored after loss, corruption, deletion or attack. Unlike redundancy, which maintains a current copy and therefore mirrors any change including a harmful one, a backup preserves an *earlier* state to which the system can return.

Because a backup is a snapshot of the past, it survives exactly the events RAID cannot: the accidental deletion, the corruption and above all the ransomware that encrypts live data and mirror alike. A widely taught rule is the *3–2–1 rule*: three copies of important data, on two different kinds of media, with one copy off-site or, in the cloud, in a separate account or region. The off-site copy matters for the same reason off-host logs mattered in Session 10, because a backup an attacker can reach and encrypt is no backup, so ransomware protection relies on copies that are off-site and immutable. The Guidance adds the rule that addresses the most common failure in practice: its preparation checklist requires backup restoration testing at regular intervals and disaster-recovery tests at least once per year (CSA, 2024, p. 255), because a backup that has never been restored is not known to work.

Real-world case: Maersk and NotPetya, 2017

The NotPetya wiper spread through the shipping company Maersk in June 2017, destroying tens of thousands of PCs and servers within minutes and halting operations at ports worldwide. The damage was reported at roughly $300 million. The company could rebuild its network identity only because a single copy of a critical directory server had survived: that machine, in an office in Ghana, had been offline during a power cut while the malware spread. Two facts follow. Redundancy that is always connected does not stop an attack that propagates; an off-line, off-site copy saved the company, the off-site element of the 3–2–1 rule. The company rebuilt because a good copy existed and could be restored; a backup nobody can recover from is no backup.

	RAID	Backup
Keeps the service running when a disk fails	yes	no
Recovers a file deleted by mistake	no	yes
Recovers data corrupted or encrypted by ransomware	no	yes
Allows a return to an *earlier* point in time	no	yes
Protects against loss of the whole site	no	yes (if off-site)

Table 11.1. RAID versus backup. They solve different problems and are used together: RAID keeps the service running through a hardware failure, backup recovers from everything else.

Table 11.1 makes the division of labour explicit. Reading down the backup column, almost everything that destroys data in practice, the accidental deletion, the corrupting bug, the ransomware, the lost data centre, is recovered by backup and none of it by RAID, because RAID copies the damage. RAID buys *uptime* through hardware failure, whereas backup buys *recoverability* from everything else. The two are therefore complementary rather than alternatives, and having RAID is never an answer to the question whether there are backups.

11.4 Redundancy the provider builds in

Redundancy runs deeper than the array built in the laboratory, because the cloud's storage is engineered for it from the bottom. The network-attached storage behind a data centre uses RAID so that the system continues to operate when a disk fails, and a failed disk is replaced while the array runs, known as *hot swapping* (Comer, 2021, pp. 103–104). Redundancy also appears above the disks: in Hadoop's distributed file system each data block is stored in redundant copies, typically three, spread across several nodes, so that the failure of a machine loses no data and computation continues on a surviving copy (Comer, 2021, pp. 156–157). The pattern is the same at every level, more than one copy on more than one device, so that any single failure is survivable, and cloud object storage expresses it as a durability figure achieved by keeping several geographically separated replicas of every object. Deletion, corruption and ransomware, however, reach through all of that provider redundancy, because replicas copy the damage as exactly as a RAID array does. Resilience is therefore layered: the provider supplies durable storage, and the customer's own backup remains the customer's responsibility.

11.5 Designing for failure

Taken together, the pieces make resilience a design stance rather than a product. Redundancy, that is, RAID beneath the storage, instances spread across availability zones and a service built to replace failed components, keeps the system serving through the failures it can absorb. Backup, off-site and tested, is the floor beneath everything, the way back when redundancy is insufficient or the loss is not a hardware failure at all. Disaster recovery planning, exercised yearly, ties the two together and answers how, and how quickly, the organisation returns if the worst happens. The immutable, use-and-replace pattern of Session 5 is thus a resilience tool as much as a security one, and the snapshot of Session 2 was the first backup.

Example 11.1 (Backing up the red thread)**.** *Threat*: the data is lost because a disk fails, a bug corrupts the database, ransomware encrypts every file, or an administrator deletes the wrong directory. *Risk*: for anything users rely on, the impact of permanent loss is severe, and several of these causes are not rare. *Controls*, correctly matched: RAID beneath the storage keeps the service running through a single disk failure but is useless against the deletion, corruption and ransomware, because it mirrors them. The decisive control is therefore a regular backup of

Nextcloud's data and configuration, kept off-site (a separate account or region), held immutably against ransomware, and restored on a schedule to prove that it works. The VM snapshot of Session 2 serves as a rollback point but must not be mistaken for the backup, and neither must the RAID. The threat is to availability and integrity; the control that meets it is the tested restore.

Reading for this chapter. CSA Security Guidance, Domain 11, §11.6 (Resilience, pp. 270–273) and the incident-response checklist's requirement for regular backup-restoration and disaster-recovery testing (§11.1.1, p. 255); Comer, Chapter 8 (Virtual Storage; §8.7 NAS, RAID and hot swapping, pp. 103–104) and §11.15 (block replication and fault tolerance in HDFS, pp. 156–157). The Session 11 laboratory builds a RAID array with `mdadm`; the choice of level is part of the exercise. The Microsoft Learn module is *Describe Azure storage* (redundancy options and Azure Backup).

11.6 Summary

Resilience is impact reduction: it accepts that failures and disasters will occur and ensures that the service survives or recovers, organised around a small blast radius. Its two tools must be kept apart. Redundancy, that is, RAID beneath storage (Definition 11.1), instances across availability zones and replaceable immutable images, keeps a system running through component failure, but RAID is not a backup, and RAID 0 is not even redundant. Backup (Definition 11.2) is a point-in-time copy that allows a return to an earlier state, so it survives the deletion, corruption and ransomware that redundancy copies exactly; it follows the 3–2–1 rule, off-site and immutable, and it is tested by restoring it, because an untested backup is not a control. Together, exercised through disaster-recovery testing, they form a design stance for failure. On the red thread, Nextcloud's data is backed up off-site and the restore is verified (Example 11.1). Session 12 covers what to do when an incident occurs.

Self-check

Sketch answers follow the exercises.

Exercise 11.1 (Resilience vs prevention)**.** Explain how resilience differs from the preventive controls of earlier sessions, in terms of likelihood versus impact.

Exercise 11.2 (The RAID 0 trap)**.** Why is RAID 0 a trap? Describe what it does and why it is less safe than a single disk for data of value.

Exercise 11.3 (Why RAID is not a backup)**.** State why RAID is not a backup. Give one failure RAID survives and two it does not.

Exercise 11.4 (The 3–2–1 rule)**.** What is the 3–2–1 rule, and why does the off-site copy matter for ransomware? Which control from Session 10 follows the same logic?

Exercise 11.5 (Data loss as threat–risk–control)**.** Write, in the vocabulary of Definition 1.3, the threat, risk and controls for the loss of the Nextcloud data, and identify the single control that actually guarantees recovery.

Sketch answers. (1) Preventive controls reduce the likelihood of a bad event (firewall, MFA, WAF); resilience accepts that failures will occur and reduces their impact, so that the service keeps running or recovers quickly. (2) RAID 0 stripes data across disks for speed with no redundancy, so any one disk failing destroys the whole array, making it more likely to lose data than a single disk despite its place among the redundant arrays. (3) RAID maintains a current copy, so it mirrors any change,

including harmful ones. It survives a disk failing; it does not survive a file being deleted or corrupted, or ransomware encrypting the data, all of which are replicated instantly to every disk. (4) Three copies, on two media types, one off-site (or a separate cloud account or region). The off-site copy matters because a backup the attacker can reach can be encrypted too; the same logic as shipping logs off-host in Session 10: a safeguard under the attacker's control is no safeguard. (5) Threat: data lost through disk failure, corruption, ransomware or accidental deletion. Risk: severe impact of permanent loss; several causes not rare. Controls: RAID for hardware failure, snapshots for rollback, and, as the one that guarantees recovery, a regular, off-site, immutable backup that is tested by restoring it.

Chapter 12

Incident response, BCM, risk and compliance

Two topics remain. The first is what to do when prevention fails and an attack succeeds: incident response, the organised handling of the breach that the whole course has been arranged to make less likely and, since Session 9, to detect. The second is the frame above every individual control, namely risk, audit and compliance, the level at which an organisation answers for its security to its management, to regulators and to the people whose data it holds.

12.1 Events, incidents and breaches

Response begins with classification, because not everything that happens is an emergency. Session 9 separated events from alerts, and the CSA Security Guidance extends this into a three-level scale (CSA, 2024, p. 254).

Definition 12.1 (Event, incident, breach)**.** An **event** describes any observable occurrence, most of them benign. An **incident** describes an event that violates security policy or threatens the confidentiality, integrity or availability of a system and demands a response. A **breach** describes a successful penetration or circumvention of controls leading to unauthorised access to, or extraction of, data. All incidents are events; not all events are incidents; and only some incidents become breaches.

The scale sets the response: an event is logged, an incident is handled, and a breach may trigger legal and regulatory obligations to notify. Getting the classification right, neither escalating every anomaly nor dismissing a real intrusion as noise, is the first judgement of response, and it depends on the monitoring and trustworthy logs of Sessions 9 and 10.

12.2 The incident response lifecycle

For handling incidents the Guidance adopts the lifecycle of NIST SP 800-61 (CSA, 2024, p. 254).

Definition 12.2 (Incident response lifecycle)**.** The **incident response lifecycle** (NIST SP 800-61) describes four phases: *Preparation* (build the team, plans, tools and baselines before anything happens); *Detection & Analysis* (identify the incident and understand its scope); *Containment, Eradication & Recovery* (stop the damage, remove the attacker, restore service); and *Post-Incident Analysis* (learn, and feed the lessons back into preparation). It is a cycle: each incident improves readiness for the next.

The Guidance's checklist gives each phase its cloud-specific content, and almost every item draws on a control from an earlier session (CSA, 2024, pp. 255–256). *Preparation* covers a team

with assigned roles, a communication plan, and responder access to environments and tools. It also covers internal documentation including a network traffic baseline (Session 2), proactive scanning and monitoring (Session 9), audit logs, snapshots and forensics capabilities (Sessions 2 and 10), and regular backup restoration testing with disaster-recovery tests at least once per year (Session 11). *Detection & Analysis* uses alerts from CSPM, SIEM, workload protection and network monitoring, validates them to reduce false positives, estimates the scope and assigns an incident manager; it then builds a timeline of the attack from the logs, which is why log integrity mattered. *Containment, Eradication & Recovery* is where the cloud differs most. Containment treats IAM and the management plane as the top priorities, isolating identities and workloads, taking systems offline, and weighing data loss against availability (CSA, 2024, pp. 267–268). Because federated identity separates authentication from authorisation, a blocked user may still hold a valid session token, so containment may require actions on both the identity provider and the relying party. Eradication then removes the attacker from the management plane permanently through credential rotation, additional policy conditions and MFA (Session 7) (CSA, 2024, pp. 268–269). In the cloud, replacing a resource is usually simpler than expelling an attacker from it, so ephemeral resources are wiped and replaced rather than fixed. Old images, old function code and old infrastructure definitions are deleted as well, because a careless redeployment of any of them could let the attacker back in. Recovery deploys hardened versions from IaC and automation, after analysing all images, resources and templates for remaining backdoors (CSA, 2024, p. 269). This form of eradication is easier for cloud-native applications and harder under lift-and-shift (Session 5). Throughout, incident data is captured under chain of custody (Session 10). Lastly, *Post-Incident Analysis* asks what should have gone better, that is, whether the attack could have been noticed earlier, which data was missing when it was needed, and whether the process itself has to change (CSA, 2024, pp. 255, 269–270), and the answers go back into preparation.

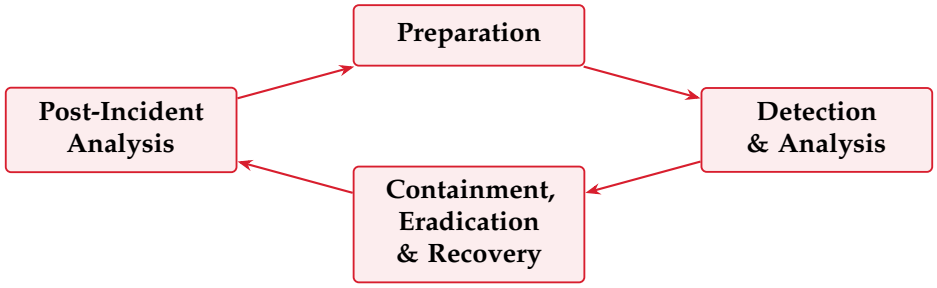

Figure 12.1. The NIST incident response lifecycle. A cycle rather than a line: each incident's lessons feed back into preparation for the next.

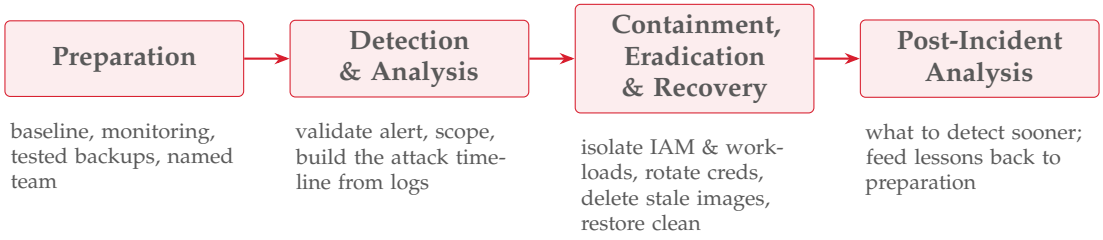

Figure 12.2. The four phases with their cloud-specific work. Almost every action draws on a control built in an earlier session: preparation on Sessions 2, 9 and 11; the timeline on Session 10; containment on Sessions 5 and 7.

Example 12.1 (An incident on the red thread)**.** The monitoring of Session 9 raises an alert: a canary file in Nextcloud was accessed, and the management plane shows an unfamiliar administrative action. *Preparation* has already provided the alert, the off-host logs and a tested

backup. *Detection & Analysis*: the alert is validated, the off-host logs are pulled, and a timeline is built: a phished administrator credential (Session 7), used from an unusual location. *Containment*: the account is disabled, its sessions revoked, the affected workload isolated. *Eradication & Recovery*: all credentials are rotated, MFA enforced, any images or configuration the attacker may have altered deleted, and Nextcloud restored from the clean backup of Session 11, whose restore was tested. *Post-Incident Analysis*: MFA was not enforced on every administrative path; this goes back into preparation. Every phase uses a control an earlier session built.

12.3 Resilience and business continuity

Incident response overlaps with a broader question the Guidance places alongside it: how the organisation keeps operating through disruption at all. The Guidance expects responders to work together with the people who keep the business running, and it reminds management, the legal department and the compliance function that a cloud incident gives each of them a part to play (CSA, 2024, pp. 257–258). *Business continuity management* (BCM) is this field. Where the resilience of Session 11 protects the technical service, continuity protects the business function the service supports, and two targets quantify it. The *recovery time objective* (RTO) states how quickly a service must be back, and the *recovery point objective* (RPO) states how much recent data the organisation can afford to lose; together they set how often backups are taken and how fast a restore must be. Continuity planning also means training staff so that they can fall back to a spare system, or to paper, while the main system is down, so that a technical failure does not become an operational collapse. Resilience, backup, disaster recovery and continuity thus form one line: keep the service up if possible, recover it fast if not, and keep the business running either way.

12.4 Risk, audit and compliance

The final step is the governance layer, because a security programme is not a set of controls but a managed response to *risk*, judged against obligations on which the organisation can be *audited*. The CSA Security Guidance treats this in Domain 3.

Definition 12.3 (Risk register)**.** A **risk register** describes a repository of identified risks, their severity, and the actions taken to manage each. It makes risk management systematic, allowing an organisation to prioritise its effort and to align mitigation with its declared *risk appetite* rather than reacting case by case (CSA, 2024, pp. 66, 78–79).

The register is the core of *governance, risk and compliance* (GRC), and the Guidance names further non-technical tools around it (CSA, 2024, pp. 78–79). The shared-responsibility model is applied by the customer to each service. *Contracts* fix the legal division of responsibility and thereby make the Session 1 model binding. A *cloud register* lists the providers and deployments the organisation depends on (CSA, 2024, pp. 66–67). Monitoring and audit frameworks provide continuous assurance that controls work (CSA, 2024, pp. 75–77). One idea is specific to the cloud, *compliance inheritance*: a customer using a compliant provider's infrastructure inherits that control set for the infrastructure layer, but the compliance of whatever the customer builds on top remains the customer's own task (CSA, 2024, p. 73), which is the shared-responsibility model in the language of audit. Automation serves governance too, because cloud infrastructure is defined in code, so its configuration is a document that can be reviewed, versioned and proven; the practice that shifted security left in Session 8 therefore also supports compliance (CSA, 2024, pp. 80–81).

For a Norwegian institution the regulatory frame is European. The CSA text is US-centred, though it lists DORA among its examples (CSA, 2024, p. 71), so the course situates its GRC

content where its students will work. The EU *NIS2* directive raises cybersecurity and incident-reporting obligations across essential and important sectors; *DORA* imposes operational-resilience and third-party-risk requirements on the financial sector; and *ISO 22301* is the international standard for business continuity management. Students are not expected to master these instruments, only to recognise that the technical work of the previous eleven sessions exists inside a legal and regulatory frame, in which a risk register, an audit trail, an incident-notification plan and a tested continuity capability are the form security takes when an organisation must answer for it.

Two observations from Comer (2021) mark the edges of what any control set can reach. First, some cloud risks are structural (Comer, 2021, p. 242). *Side channels* leak information between tenants sharing physical hardware through indirect signals rather than any explicit access, and *back doors* are hidden means of access left in software or hardware. Neither is stopped by a firewall or a password, so these are exactly the risks a register exists to record and to accept knowingly. Secondly, the provider is not only a source of risk but a partner in managing it (Comer, 2021, pp. 242–243). Traditional IT kept outsiders out on principle, whereas a cloud tenant depends on the provider, sets its own security policy and relies on the provider to help carry it out. A tenant with several providers has to make sure that the same policy holds on each of them. This is compliance inheritance seen from the other side. Governance therefore covers not only the controls an organisation operates itself but also the choice, assessment and contracting of the provider whose controls it inherits, together with a clear view of the residual risks that remain its own.

Reading for this chapter. CSA Security Guidance, Domain 11, §11.1 (Incident Response and the lifecycle checklist, pp. 254–256), §11.2 (Preparation, pp. 256–261), §11.3 (Detection & Analysis, pp. 261–267), §11.4 (Containment, Eradication & Recovery, pp. 267–269) and §11.5 (Post-Incident Analysis, pp. 269–270); Domain 3, §3.1 (Cloud Risk Management, including the cloud register, pp. 56–68), §3.2 (Compliance & Audit, including compliance inheritance, pp. 68–78) and §3.3 (GRC Tools & Technologies, pp. 78–81). Comer, Chapter 17, §§17.11–17.12 (back doors, side channels and other concerns, and cloud providers as partners, pp. 242–243). The Microsoft Learn module is *Describe Azure governance* (Azure Policy, Microsoft Purview compliance).

12.5 Summary

When prevention fails, response takes over, beginning with the classification of events, incidents and breaches (Definition 12.1) that sets the level of reaction. The incident response lifecycle (Definition 12.2), with the phases Preparation, Detection & Analysis, Containment/Eradication/Recovery and Post-Incident Analysis, gathers the course into one procedure, because the baseline, monitoring, snapshots, off-host logs, MFA and tested backups of earlier sessions are exactly what each phase uses. The cloud-specific priorities are containing the management plane, rotating credentials, deleting stale images and recovering from clean definitions (Example 12.1). Above the incident sits business continuity, with RTO and RPO, spare systems and trained staff, which keeps the business function alive through disruption. Above the technical layer sits governance, risk and compliance: the risk register (Definition 12.3) and the GRC tools around it, compliance inheritance from the provider, the auditability of infrastructure as code, and the regulatory frame (NIS2, DORA, ISO 22301) within which a Norwegian organisation operates. Session 13 assembles the layers into one argument.

Self-check

Sketch answers follow the exercises.

Exercise 12.1 (Event, incident, breach)**.** Distinguish an event, an incident and a breach, and state which may trigger legal notification obligations.

Exercise 12.2 (The four incident-response phases)**.** Name the four phases of the incident response lifecycle and, for each, one control from an earlier session that it relies on.

Exercise 12.3 (Why rebuild rather than patch)**.** In cloud containment and eradication, why rotate credentials and delete old images and IaC rather than simply patching the running system?

Exercise 12.4 (RTO and RPO)**.** What do RTO and RPO measure, and how do they determine how often backups are taken and how fast a restore must be?

Exercise 12.5 (Risk register and compliance inheritance)**.** What is a risk register, and what is compliance inheritance? Relate the latter to the shared-responsibility model of Session 1.

Sketch answers. (1) An event is any observable occurrence; an incident is an event that violates policy or threatens CIA and demands a response; a breach is a successful compromise leading to unauthorised data access or extraction. A breach may trigger legal or regulatory notification. (2) Preparation (baseline, monitoring, tested backups); Detection & Analysis (trustworthy off-host logs for the timeline); Containment/Eradication/Recovery (MFA and IAM from Session 7, immutable images from Session 5, backups from Session 11); Post-Incident Analysis (feeds lessons back into preparation). (3) Because the attacker may hold stolen credentials and may have left back doors in images, serverless code or IaC; rotating credentials revokes their access, and rebuilding from clean definitions removes hidden footholds a patch would leave in place. (4) RTO is how quickly a service must be restored; RPO is how much recent data may be lost. RPO sets backup frequency (lose at most that much); RTO sets the required restore speed and the recovery design. (5) A risk register lists identified risks, their severity and mitigation actions, aligning effort with risk appetite. Compliance inheritance means inheriting the provider's control set for the layers it operates while remaining responsible for the compliance of one's own software: the shared-responsibility model expressed in the language of audit.

Chapter 13

The integrated case

This chapter introduces no new material. It gathers the controls of Sessions 2 to 12 into one picture, maps them onto the NIST Cybersecurity Framework 2.0, and makes the argument, layer by layer, that the hardened service is defensible.

13.1 The hardened stack

Each session added one layer to the same service, and each layer answered a threat with a control and a reason. Session 2 set up a virtual machine running Linux and recorded its *baseline*, the picture of normal that every later detection measures against. Session 3 put the service under load and covered denial of service. Session 4 closed the ports with a *firewall* under default-deny. Session 5 deployed Nextcloud as a *container*, choosing the cloud-native, immutable pattern. Session 6 wrapped the transport in *TLS*, and Session 7 hardened *identity* with MFA and least privilege. Session 8 shielded the application with a *WAF* and built security into the pipeline with DevSecOps. Session 9 turned the baseline into continuous *monitoring*, and Session 10 protected the *logs* off-host and tamper-evident, so that a compromise cannot erase its own trace. Session 11 made the data survivable with *backup* and redundancy, tested by restoring, and Session 12 prepared the *response* to the incident that gets through, inside a frame of risk, audit and compliance.

These layers are independent by design, and that is what makes them one system rather than a list. The firewall catches what the WAF would not, the WAF catches what secure coding missed, MFA catches the stolen password, monitoring catches what all of them let through, and backups recover what nothing prevented. Defence in depth (Session 4) is therefore not a slogan here but the structure of what was built.

13.2 Mapping to the NIST Cybersecurity Framework

To show that the picture is complete rather than merely long, it is mapped onto a recognised framework. The NIST Cybersecurity Framework 2.0 (2024) has three levels. At the top are six *Functions*: Govern, Identify, Protect, Detect, Respond and Recover. Each Function divides into *Categories*, groups of related outcomes; Protect, for example, has one Category for identity and access control, another for data security and another for platform security. Each Category divides again into *Subcategories*, the specific, testable outcomes an organisation achieves. In total CSF 2.0 defines 22 Categories and 106 Subcategories. Govern is drawn at the centre, because it sets the strategy and policy on which the other five depend, and whereas Govern, Identify, Protect and Detect run continuously, Respond and Recover stand ready for the incident.

The Categories are where completeness is tested. Claiming to protect a service is easy, whereas the Categories ask whether identity, data, the platform and infrastructure resilience are

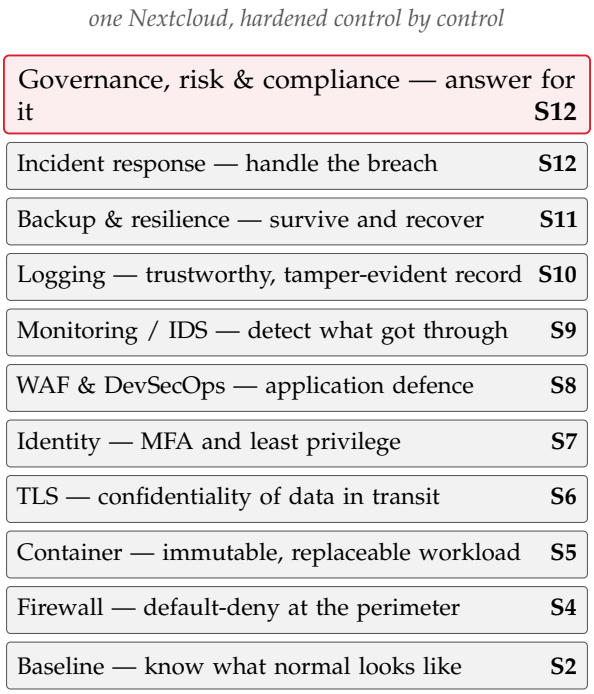

Figure 13.1. The hardened service. Each session added one independent layer to the same Nextcloud; together they form one defensible system.

each protected. Table 13.1 therefore places each Function's Categories beside the sessions that populate them.

Reading down the table is the test of the word defensible. It does not mean that the service cannot be attacked; it means that the protection is complete across the framework at the level of Categories, and that for each Category the control, the threat it meets and the reason it belongs can be named. An empty Category is a real gap, because a service with no entry under Detect is blind and one with nothing under Recover cannot come back. A mature programme works down to the Subcategories; for this course, a control in every Category, defended in the threat–risk–control vocabulary of Session 1, is the standard.

13.3 The argument

The course ends where it began. Behind the metaphor of the cloud stand machines, networks, identities and data, each with a threat, each met by a control chosen because the threat was understood. Security is therefore a method rather than a product: name the threat, judge the risk, choose the control that fits, state why, and add the next layer, because no single control is sufficient. The method has been applied eleven times to one service, and it transfers to any other.

One further observation explains why the task is never simply finished. Comer (2021) closes his book with the complexity of cloud-native systems (Comer, 2021, pp. 247–249). A cloud-native application is a large *distributed* system, and such systems are complex by nature, however carefully they are built: copies of the same data drift apart, the number of instances can explode under load, network delays have effects nobody planned for, and processes can end up waiting for each other forever. Security lives inside that complexity. This is why the defensible system is a set of independent layers rather than a single mechanism, and why monitoring assumes that prevention will fail. It is also why the question whether a system is secure never has a permanent yes as its answer, only a position that must be argued again as the system changes.

Function	Categories (CSF 2.0)	Where this course populates it
Govern (GV)	Organizational Context; Risk Management Strategy; Roles, Responsibilities & Authorities; Policy; Oversight; Cybersecurity Supply Chain Risk Management	Risk register, shared-responsibility model, the NIS2/DORA/ISO 22301 frame, and supply-chain risk via the SBOM (S12, S8)
Identify (ID)	Asset Management; Risk Assessment; Improvement	Data classification (S6); asset and dependency inventory / SBOM (S8); risk assessment and the register (S12)
Protect (PR)	Identity Management, Authentication & Access Control; Awareness & Training; Data Security; Platform Security; Technology Infrastructure Resilience	IAM, MFA and least privilege (S7); TLS and data protection (S6); the firewall and segmentation (S4); WAF and secure development (S8); platform hardening: baseline, containers, patching (S2, S5); backup and resilience (S11)
Detect (DE)	Continuous Monitoring; Adverse Event Analysis	Monitoring, IDS/IPS and telemetry (S9); trustworthy, tamper-evident logs (S10)
Respond (RS)	Incident Management; Incident Analysis; Incident Response Reporting & Communication; Incident Mitigation	The incident response lifecycle: containment, eradication, analysis and reporting (S12)
Recover (RC)	Incident Recovery Plan Execution; Incident Recovery Communication	Tested backup restoration, disaster recovery and business continuity (S11, S12)

Table 13.1. The course mapped onto the six CSF 2.0 Functions and their Categories. Every Function is populated, and so is every Category: identity, data, platform and infrastructure resilience each have a home under Protect, rather than Protect being a single undivided claim.

Reading for this chapter. No new domains. Students revisit the domains and chapters behind each layer as they assemble their case; Comer, Chapter 18 (Controlling the Complexity of Cloud-Native Systems, pp. 247–249), is the intellectual close. The NIST Cybersecurity Framework 2.0 (2024) is the mapping framework; its six Functions and their Categories should be known by name.

13.4 Summary

One Nextcloud has been hardened across thirteen sessions into a single defensible system whose layers (baseline, firewall, container, TLS, identity, WAF, monitoring, logging, backup, response, governance) are independent by design, so that each catches what the others miss. Mapped onto the six Functions of the NIST Cybersecurity Framework 2.0 and their Categories, the protection is complete rather than merely long, which is what defensible means here. The method is the same at every layer: name the threat, judge the risk, choose the control that fits, state why, and add the next layer. The cloud remains somebody else's computer; what the course adds is the ability to argue, layer by layer, why the service running on it is defensible.

Closing exercise

Take the group's own hardened service and, for each of the six NIST CSF 2.0 Functions (Govern, Identify, Protect, Detect, Respond, Recover), name one control implemented, the threat it meets, and one sentence on why it belongs there. Better still, go one level finer and place each control under a *Category* of Table 13.1. If every Function has at least one entry and each can be defended in the threat–risk–control vocabulary of Chapter 1, the defence is complete across the framework. Where a Function or a Category is thin, that is a gap in the defence rather than in the exercise, and finding it now is the purpose of the exercise.

Reading list

This table gathers the reading named at the end of each chapter so that the whole plan can be seen at a glance. Each row lists the *core* reading for that session, followed by *further* or optional deepening; the per-chapter blocks give the same references with more context. Page numbers refer to the editions below; for the CSA Security Guidance they are the printed page numbers in the document's footer. The weekly Microsoft Learn module is a graded companion, not a substitute for the reading.

Core texts. *CSA* — Cloud Security Alliance (2024), *Security Guidance for Critical Areas of Focus in Cloud Computing v5*. *Comer* — D. E. Comer (2021), *The Cloud Computing Book: The Future of Computing Explained*, Chapman & Hall/CRC. *Jøsang* — A. Jøsang (2024), *Cybersecurity: Technology and Governance*, Springer, ISBN 978-3-031-68483-8. *Tanenbaum and Bos* — A. S. Tanenbaum and H. Bos (2015), *Modern Operating Systems*, 4th edition, Pearson; drawn on for operating-systems foundations in Sessions 2 and 7.

S	Topic	Reading (core, then further)	MS Learn
1	Concepts, CIA, threat–risk–control, service models	CSA Domain 1 (pp. 10–27); Comer Ch. 1–3 (pp. 5–32) and §17.2 (pp. 233–234); Jøsang Ch. 1 (pp. 1–24).	Describe cloud concepts
2	Data centres, virtualisation and Linux	Comer Ch. 4 (pp. 37–51), Ch. 5 (pp. 55–68) and Ch. 3 (pp. 25–32); CSA D7 §7.1 (pp. 147–153); Tanenbaum and Bos §7.2–7.3 (pp. 474–477), Ch. 10–11; Jøsang Ch. 3 (pp. 43–62).	Describe Azure architecture & services
3	Web services, load and denial of service	Comer §1.5 (pp. 8–9), Ch. 2 (pp. 15–22), §14.2–14.7 (pp. 195–200); CSA D7 §7.2 (pp. 153–162); Jøsang Ch. 2 (pp. 25–42).	Describe Azure compute & networking
4	Networking and firewall controls	Comer Ch. 7 (pp. 87–96) and §17.3–17.5 (pp. 235–237); CSA D7 §7.2 (pp. 153–162), §7.4 (pp. 168–176); Jøsang Ch. 6 (pp. 117–142).	Describe Azure networking services
5	Cloud-native versus lift-and-shift	Comer Ch. 5 (pp. 55–68), Ch. 6 (pp. 71–83), Ch. 10 (pp. 127–140), §11.3 (p. 146); CSA D8 §8.1 (pp. 177–182), D7 §7.1.5 (pp. 152–153); Jøsang Ch. 11 (pp. 243–270).	Describe Azure compute (containers)
6	TLS and data in transit	CSA D9 §9.1.2 (pp. 210–211), §9.2.3–9.2.5 (pp. 216–221) and p. 229; Comer §17.9 (pp. 240–241); Jøsang Ch. 4–5 (pp. 63–116).	Describe Azure security
7	Hardening and access control	CSA D5 §5.1–5.5 (pp. 103–123); Comer §17.5–17.7 (pp. 237–239); Tanenbaum and Bos §9.3 (pp. 602–604); Jøsang Ch. 8–9 (pp. 165–214).	Describe Azure identity

continued on next page

S	Topic	Reading (core, then further)	MS Learn
8	DevSecOps, WAF and secure applications	CSA D10 §10.1/§10.4/§10.5 (pp. 230–252) and D8 §8.1.4–8.1.5 (pp. 181–182); Comer Ch. 15 (pp. 207–214) and §9.10–9.11 (pp. 121–122); Jøsang Ch. 11 (pp. 243–270).	Azure Application Gateway & WAF
9	Security monitoring and IDS	CSA D6 §6.1–6.4 (pp. 124–145); Comer §13.3–13.4 (pp. 182–183) and §17.8 (pp. 239–240); MITRE ATT&CK; Jøsang Ch. 16 (pp. 337–354).	Describe Azure monitoring
10	Logging and log integrity	CSA D6 §6.1.1 (pp. 125–126), §6.2 (pp. 127–133), §6.3 (pp. 133–135); CSA D11 §11.1.1 (pp. 254–256), §11.3.4 (pp. 265–267); Jøsang Ch. 16 (pp. 337–354).	Azure Monitor & Log Analytics
11	Resilience, RAID and backup	CSA D11 §11.6 (pp. 270–273), §11.1.1 (p. 255); Comer §8.7 (pp. 103–104) and §11.15 (pp. 156–157).	Describe Azure storage
12	Incident response, BCM, risk and compliance	CSA D11 §11.1–11.5 (pp. 254–270); CSA D3 §3.1–3.3 (pp. 56–81); Comer §17.11–17.12 (pp. 242–243).	Describe Azure governance
13	The integrated case	No new domains; Comer Ch. 18 (pp. 247–249); NIST Cybersecurity Framework 2.0.	—